Código:FT-SL-003
Versión:001
Fecha:21/12/2023

FORMATO DE DESARROLLO E INTEGRACIONES

FT-SL-003-Módulo_Pago

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, martes, 15 de septiembre de 2026

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 6
1.2 Detallar funcionalidades 6
1.3 Observaciones y/o Comentarios..... 59
2. CASOS DE PRUEBAS UNITARIAS 60

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO Ap

1.1 Descripción del Requerimiento:

1.1.1 Requerimiento

Realizar todo el proceso de pagos de los créditos realizados con el cupo de credi10, garantizando el adecuado cobro de los diferentes rubros correspondientes a mora, aval, interés, los impuestos correspondientes y el capital. El requerimiento se divide en dos objetivos específicos.

1. Realizar calculo a la fecha de los diferentes montos pendientes a pagar.
2. Registrar pagos, generando un comprobante.

Ver el requerimiento en el link: FT-SL-003-Módulo_Pago-V5

1.1.2 Flujo del Proceso

Comentado [HA1]: Agregar mockup o capturas del flujo del proceso

- *Pago en tienda*



Pantalla 2



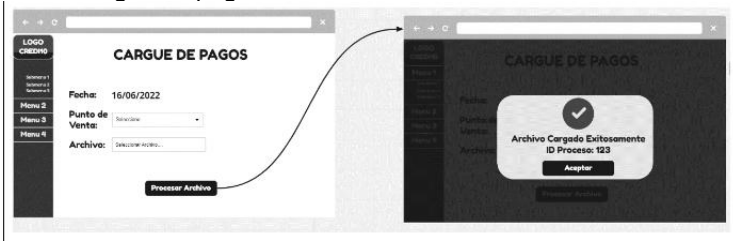
- *Pago por consignación*



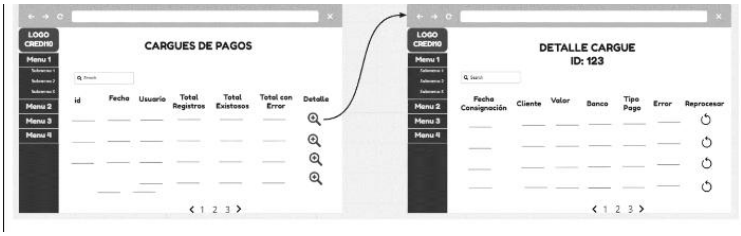
Pantalla 2



- *Cargue de pago*



- *Seguimiento del cargue de pagos*



1.2 Detallar componentes existentes

Nombre del Componente	Tipo
SOLUCION	
Creditos.Web.Admin	Controladores y vistas implementadas para cumplir con el propósito de informar el valor del pago y recibir el valor que quieran aplicar.
Creditos.BLL	Proyecto que contiene las clases y métodos que implementan la lógica de negocio de la aplicación, esta biblioteca de clases será el encargado de calcular los pagos, procesarlos, generar y enviar el ticket. Este proyecto se integra con el proyecto Web.Admin, el proyecto DAL y el proyecto DTO.
Creditos.DAL	Proyecto encargado de tener el acceso y la interacción a la base de datos C10_Credi10NET de la solución.
Creditos.DTO	Proyecto que contiene los diferentes modelos usados en toda la aplicación.
Rotativa	Herramienta de línea de comandos para representar HTML en PDF, encargado de crear los tickets de impresión del pago.
BASE DE DATOS	
C10_Credi10NET	Tablas y Procedimientos almacenados

1.3 Detallar funcionalidades

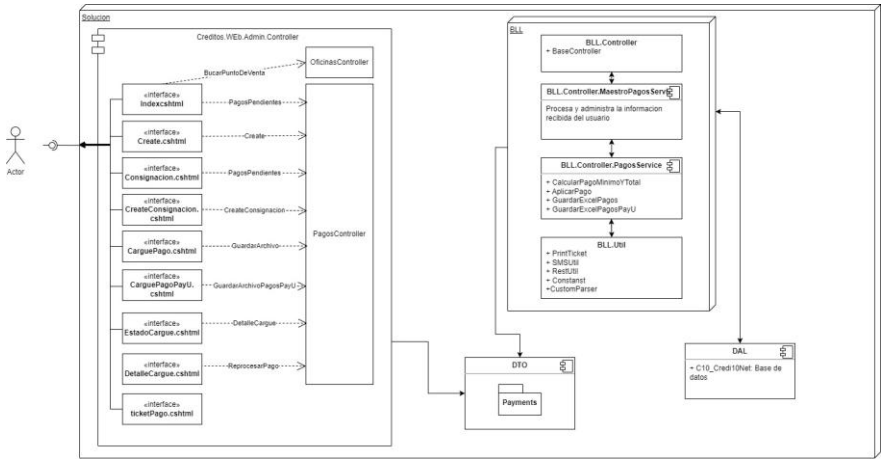
1.3.1 Diagrama de Arquitectura

Comentado [HA2]: Usar para estos manuales, mejor diagramas de componentes ya que por ser funcionalidades internas los diagramas de arquitectura dan una visualizacion muy general

Opcional usar: https://app.diagrams.net/

https://www.youtube.com/watch?app=desktop&v=JF7bH5Xo6ys

https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/diagrama-de-componentes/

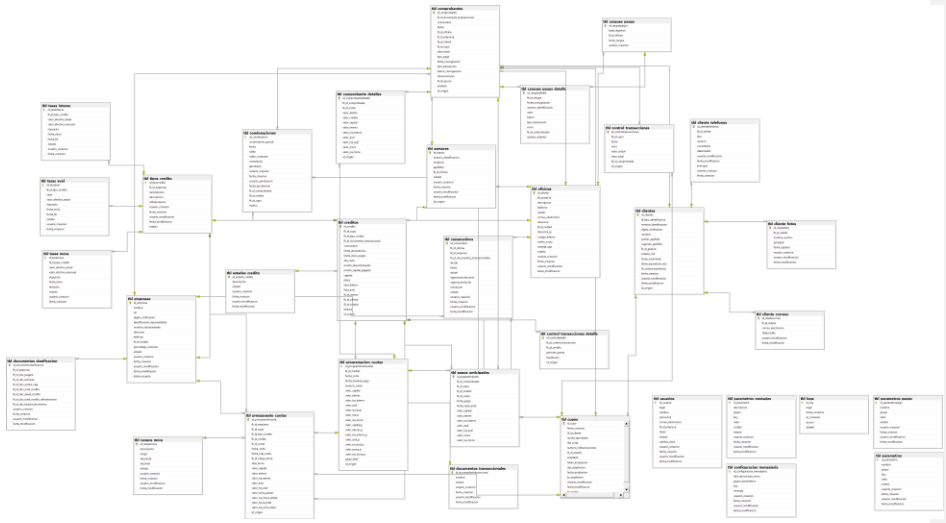


Flujos realizados para calcular el pago mínimo y total de un cliente y para aplicar un pago en general.

- Diagramas De Flujo

1.3.2 MER

Comentado [HA3]: Agregar el MER de las tablas que tienen que ver con el proceso



1.3.3 Parametrizaciones

Comentado [HA4]: Agregar el listado de los parametros generales que se usan en el proceso.

Descripción	Grupo	Valor
Lista de bancos permitido para la consignación.	Banco	BANCOLOMBIA
Lista de bancos permitido para la consignación.	Banco	AV VILLAS
Lista de bancos permitido para la consignación.	Banco	WOMPI
Medio de pagos permitidos para registrar el movimiento del RC.	MedioPago	TX
Medio de pagos permitidos para registrar el movimiento del RC.	MedioPago	CG
Medio de pagos permitidos para registrar el movimiento del RC.	MedioPago	WP
Tipo de pago que se podrá hacer en el sistema para los diferentes procesos.	TipoPago	Consignación
Tipo de pago que se podrá hacer en el sistema para los diferentes procesos.	TipoPago	Físico
Tipo de pago que se podrá hacer en el sistema para los diferentes procesos.	TipoPago	Refinanciación
Tipo de pago que se podrá hacer en el sistema para los diferentes procesos.	TipoPago	Online
Mensaje de las tiendas que ya alcanzaron su límite de recaudo	Pagos	Lo sentimos, ya se alcanzó el límite de recaudo
Mensaje al ingresar un valor no permitido para el pago	Pagos	El valor del pago está fuera del rango permitido
Mensaje mostrado cuando el valor del pago ingresado	Pagos	El valor ingresado supera el límite del recaudo permitido

supera el límite de recaudo de la empresa aliada.		a la empresa. Valor máximo actual es de ${0}
Mensaje mostrado cuando el pago se realiza correctamente	Pagos	Pago Realizado
Mensaje cuando hubo un problema al intentar generar el pago en tienda o consignación.	Pagos	Lo sentimos, error procesando el pago. Inténtalo de nuevo más tarde
Mensaje cuando el archivo del cargue de pagos se subió e inicio el proceso de los pagos correctamente	Pagos	Archivo cargado exitosamente. ID Proceso {0}
Mensaje si el archivo del cargue de pagos presenta algún tipo de error que el sistema detecto.	Pagos	Sin registros válidos. Por favor sube otro archivo o verifica que no hallan consignaciones cargadas previamente.
Mensaje mostrado si en el tiempo transcurrido desde la consulta y la aplicación del pago el valor del mismo pudo haber cambiado por un movimiento realizado.	Pagos	Hubo un movimiento de crédito que pudo haber cambiado el valor del pago. Por favor repite el proceso de consulta
Oficina para recibir pagos web	Pagos	298
SMS - Mensaje Pago Realizado	SMS	Estimado cliente CREDI10 acabamos de recibir su pago por valor de ${0} con fecha de {1}.

1.3.4 Proceso

Cuando un usuario inicia sesión en la aplicación web de credi10 este va a tener un perfil asignado el cual le concederá permisos para ver y realizar diferentes operaciones dentro del sistema. Para acceder a la vista de pagos en tienda el usuario conectado debe si o si tener algún permiso para poder visualizar esta vista.



Esta vista tiene como ruta https://www.credi10.com.co/Pagos, si se intenta acceder a ella sin iniciar sesión antes será redirigido a la vista de Login Login (credi10.com.co) y si ya ha iniciado sesión y el usuario no tiene los permisos adecuados e intentará acceder directamente por medio del link, el acceso le será negado y mostrará un error en la vista.

Esto ocurrirá con todas las vistas que requieran un mínimo de permisos para poder ser visualizadas, en todo el módulo de pagos funcionará así. A continuación, se describirá el funcionamiento de todo el proceso para generar un pago desde el ingreso a la primera pantalla.

- **Pagos En Tienda**

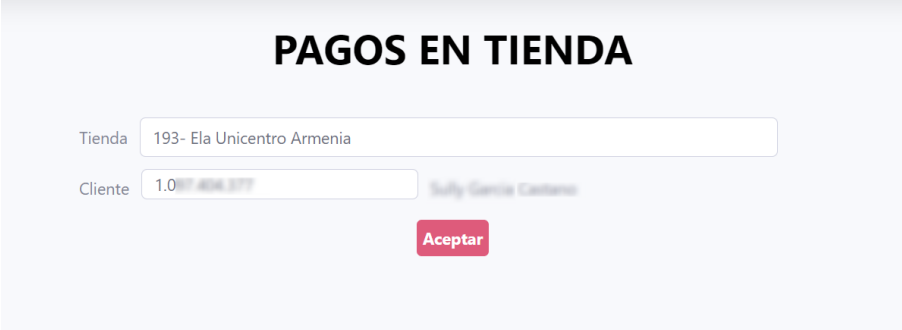
```
// GET: Payment
[HttpGet]
0 referencias
public ActionResult Index()
{
    ViewBag.Key = SecurityUtil.GenerateToken();
    ValidateSession(true);
    var response = Admin.GetOficinaIP(SecurityUtil.GetIP());
    ViewBag.Response = response.Completed;
    ViewBag.OficinaIP = response.Data;
    ViewBag.Oficinas = Param.ListaDesplegableOficinas(SessionUser.IdEmpresa);
    return View();
}
```

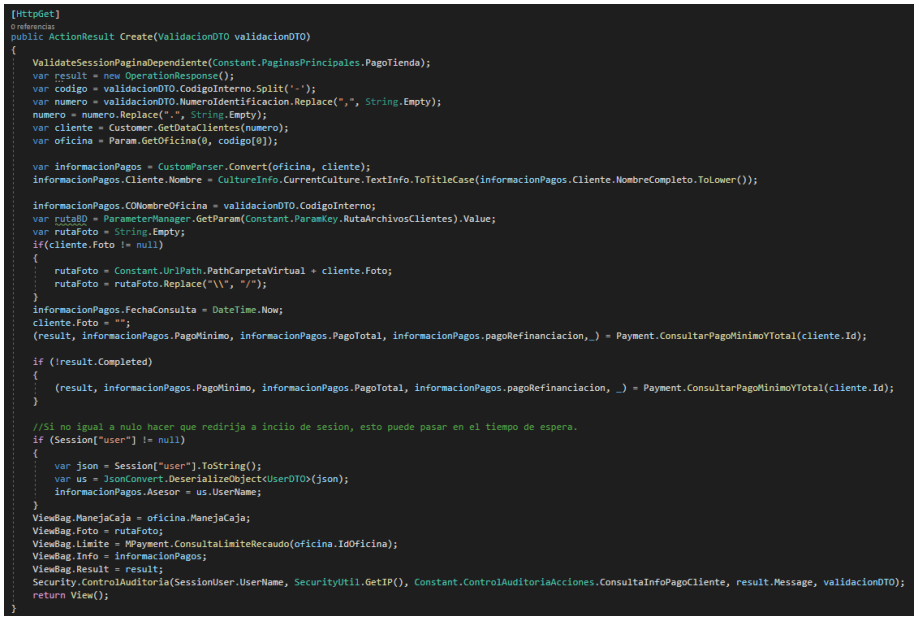
Al ingresar al índex de pagos en tienda (https://www.credi10.com.co/Pagos) llamara al método ValidateSession explicado aquí, que garantiza si el usuario tiene permisos de ingresar al recurso, obtiene y valida la dirección de la ip de conexión para saber si es una tienda específica para que cuando cargue la vista la tienda ya este seleccionada y mejorar la experiencia de usuario, si no hay coincidencias en las direcciones IP la vista cargara una lista desplegable de diferentes tiendas que no cuentan con una IP fija o en el mismo campo se le dará la opción de ingresar el código interno (C.O.) de la tienda.

La vista de pagos en tienda tendrá la validación de si debe tener seleccionado un punto de venta (oficina) o si debe cargar la lista desplegable de oficinas, el código que realiza esto en la vista es el siguiente:

```
@if (ViewBag.Response)
{
    <script>
        $(document).ready(function (e) {
            $('#CodigoInterno').val("@ViewBag.OficinaIP");
        });
    </script>
    validacionOficina = true;
}
```

Así sería la vista de pagos en tienda al ya tener seleccionado una tienda y de haber ingresado una cedula que tiene una cuenta por pagar a credi10. Si las validaciones pasan se activará el botón de Aceptar para que puedan dirigirse a la otra vista donde traerá la información del pago y del cliente.





```
[HttpGet]
0 referencias
public ActionResult Create(ValidacionDTO validacionDTO)
{
    ValidateSessionPaginaDependiente(Constant.PaginasPrincipales.PagoTienda);
    var result = new OperationResponse();
    var codigo = validacionDTO.CodigoInterno.Split('-');
    var numero = validacionDTO.NumeroIdentificacion.Replace(",", String.Empty);
    numero = numero.Replace(".", String.Empty);
    var cliente = Customer.GetDataClientes(numero);
    var oficina = Param.GetOficina(0, codigo[0]);

    var informacionPagos = CustomParser.Convert(oficina, cliente);
    informacionPagos.Cliente.Nombre = CultureInfo.CurrentCulture.TextInfo.ToTitleCase(informacionPagos.Cliente.NombreCompleto.ToLower());

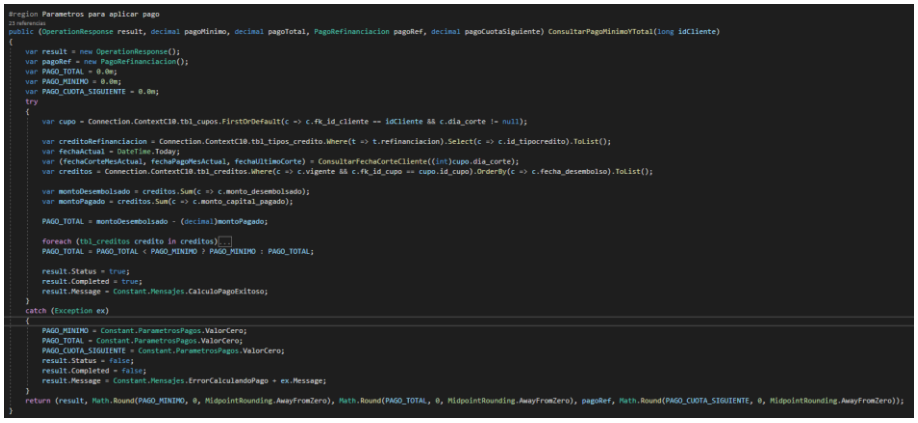
    informacionPagos.CONombreOficina = validacionDTO.CodigoInterno;
    var rutaBD = ParameterManager.GetParam(Constant.ParamKey.RutaArchivosClientes).Value;
    var rutaFoto = String.Empty;
    if(cliente.Foto != null)
    {
        rutaFoto = Constant.UrlPath.PathCarpetaVirtual + cliente.Foto;
        rutaFoto = rutaFoto.Replace("\\", "/");
    }
    informacionPagos.FechaConsulta = DateTime.Now;
    cliente.Foto = "";
    (result, informacionPagos.PagoMinimo, informacionPagos.PagoTotal, informacionPagos.pagoRefinanciacion,_) = Payment.ConsultarPagoMinimoYTotal(cliente.Id);

    if (!result.Completed)
    {
        (result, informacionPagos.PagoMinimo, informacionPagos.PagoTotal, informacionPagos.pagoRefinanciacion, _) = Payment.ConsultarPagoMinimoYTotal(cliente.Id);
    }

    //Si no igual a nulo hacer que redirija a inciio de sesion, esto puede pasar en el tiempo de espera.
    if (Session["user"] != null)
    {
        var json = Session["user"].ToString();
        var us = JsonConvert.DeserializeObject<UserDTO>(json);
        informacionPagos.Asesor = us.UserName;
    }
    ViewBag.ManejaCaja = oficina.ManejaCaja;
    ViewBag.Foto = rutaFoto;
    ViewBag.Limite = MPayment.ConsultaLimiteRecaudo(oficina.IdOficina);
    ViewBag.Info = informacionPagos;
    ViewBag.Result = result;
    Security.ControlAuditoria(SessionUser.UserName, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.ConsultaInfoPagoCliente, result.Message, validacionDTO);
    return View();
}
```

Cuando se le da clic en Aceptar mediante Ajax se hace llamado al endpoint de *Create()* enviando los dos valores ingresados en el formulario. Este método en general se encarga de traer los datos básicos del cliente, los datos de la oficina y llamará a el método *ConsultarPagoMinimoYTotal()*, dependiendo si el resultado de este cálculo es correcto o no se volverá a realizar un segundo intento para realizar dicho calculo, esto se debe para asegurar que en la segunda pantalla muestre los datos correctos y no generarles inconvenientes a las asesoras y clientas en tienda. El método será explicado en un ítem aparte por la importancia que tiene en la aplicación. También a la vista se devolverá datos puntuales de la tienda como si esta maneja caja o llego a su límite de recaudo, el cual, el valor que este se devuelva a la vista bloqueara a la tienda para que prosiga con el pago, lo que mostrara solo la información del cliente y el valor del pago.

- **Consultar Pago mínimo y total**



```
#region Parametros para aplicar pago
2 references
public (OperationResponse result, decimal pagoMinimo, decimal pagoTotal, PagoRefinanciacion pagoRef, decimal pagoCuotaSiguiente) ConsultarPagoMinimoYTotal(long idCliente)
{
    var result = new OperationResponse();
    var pagoRef = new PagoRefinanciacion();
    var PAGO_TOTAL = 0.0m;
    var PAGO_MINIMO = 0.0m;
    var PAGO_CUOTA_SIGUIENTE = 0.0m;
    try
    {
        var cupo = Connection.ContextC10.tbl_cupos.FirstOrDefault(c => c.fk_id_cliente == idCliente && c.dia_corte != null);

        var creditoRefinanciacion = Connection.ContextC10.tbl_tipos_credito.Where(t => t.refinanciacion).Select(c => c.id_tipocredito).ToList();
        var fechaActual = DateTime.Today;
        var (fechaCorteMesActual, fechaPagoMesActual, fechaUltimoCorte) = ConsultarFechaCorteCliente((int)cupo.dia_corte);
        var creditos = Connection.ContextC10.tbl_creditos.Where(c => c.vigente && c.fk_id_cupo == cupo.id_cupo).OrderBy(c => c.fecha_desembolso).ToList();

        var montoDesembolsado = creditos.Sum(c => c.monto_desembolsado);
        var montoPagado = creditos.Sum(c => c.monto_capital_pagado);

        PAGO_TOTAL = montoDesembolsado - (decimal)montoPagado;

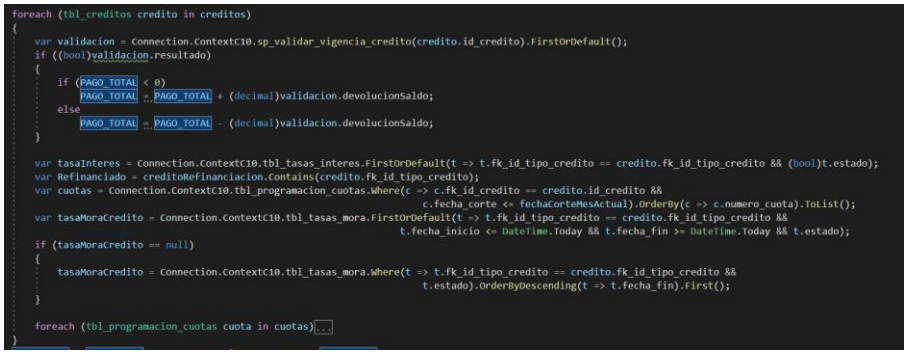
        foreach (tbl_creditos credito in creditos)...
        PAGO_TOTAL = PAGO_TOTAL < PAGO_MINIMO ? PAGO_MINIMO : PAGO_TOTAL;

        result.Status = true;
        result.Completed = true;
        result.Message = Constant.Mensajes.CalculoPagoExitoso;
    }
    catch (Exception ex)
    {
        PAGO_MINIMO = Constant.ParametrosPagos.ValorCero;
        PAGO_TOTAL = Constant.ParametrosPagos.ValorCero;
        PAGO_CUOTA_SIGUIENTE = Constant.ParametrosPagos.ValorCero;
        result.Status = false;
        result.Completed = false;
        result.Message = Constant.Mensajes.ErrorCalculandoPago + ex.Message;
    }
    return (result, Math.Round(PAGO_MINIMO, 0, MidpointRounding.AwayFromZero), Math.Round(PAGO_TOTAL, 0, MidpointRounding.AwayFromZero), pagoRef, Math.Round(PAGO_CUOTA_SIGUIENTE, 0, MidpointRounding.AwayFromZero));
}
```

El método en mención recibe como único parámetro el *idCliente* y este retornara una tupla con 5 valores diferentes, *result* que será un objeto que me indique el resultado de la operación, *pagoMinimo* que será un valor decimal que indique el pago mínimo que tenga el cliente consultado, *pagoTotal* que será un valor decimal que retorne el pago total con el que el cliente saldaría toda la deuda con credi10, *pagoRef* que será un objeto que ayudara a identificar si el cliente cuenta con una primera cuota pendiente por pagar de un crédito refinanciado y esta cuota no ha pasado por corte, el objeto contiene el valor y un status para identificar si se debe o no mostrar en la pantalla este valor adicional y por ultimo *pagoCuotaSiguiente* que será un valor decimal que proporciona un valor informativo de cuanto seria la próxima cuota a pagar.

El método se encierra en un try y catch para capturar cualquier tipo de excepción que se pueda producir durante el cálculo. El método inicia encontrando el cupo asociado al idCliente enviado y con respecto a este cupo encontrar todos los créditos asociados que se encuentren en un estado vigente, organizados de manera ascendente con respecto a la fecha de desembolso con el que fue realizado y también calcula con el método *ConsultarFechaCorteCliente()* cuales son las fechas importantes que tiene actualmente que son las de corte y pago del cliente.

Al tener la lista de créditos vigentes del cliente se calcula el monto total desembolsado y el monto total del capital que han pagado a los créditos. El valor del pagoTotal se inicia con la resta de estos dos ítems del crédito, que sería el saldo total a capital que adeuda el cliente sin tener en cuenta interés, avales, mora e IVA que serán sumados en lo que resta del método.



```
foreach (tbl_creditos credito in creditos)
{
    var validacion = Connection.ContextC10.sp_validar_vigencia_credito(credito.id_credito).FirstOrDefault();
    if ((bool)validacion.resultado)
    {
        if (PAGO_TOTAL < 0)
            PAGO_TOTAL = PAGO_TOTAL + (decimal)validacion.devolucionSaldo;
        else
            PAGO_TOTAL = PAGO_TOTAL - (decimal)validacion.devolucionSaldo;
    }

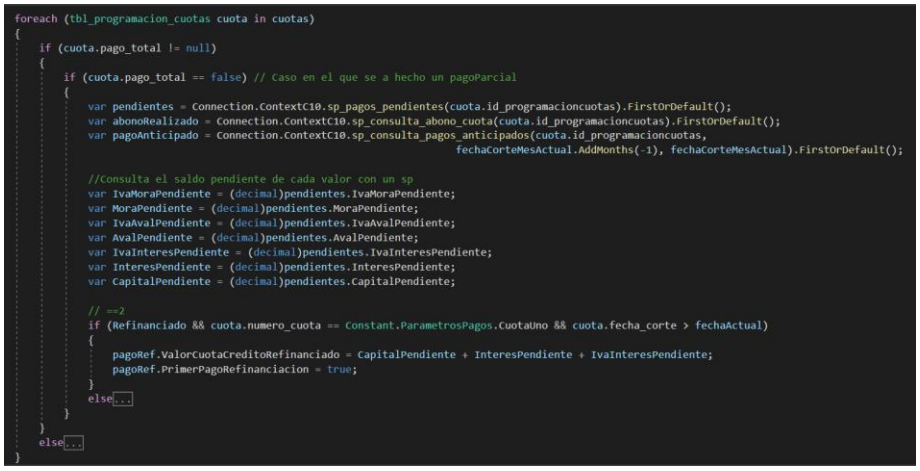
    var tasaInteres = Connection.ContextC10.tbl_tasas_interes.FirstOrDefault(t => t.fk_id_tipo_credito == credito.fk_id_tipo_credito && (bool)t.estado);
    var Refinanciado = creditoRefinanciacion.Contains(credito.fk_id_tipo_credito);
    var cuotas = Connection.ContextC10.tbl_programacion_cuotas.Where(c => c.fk_id_credito == credito.id_credito &&
                                                                          c.fecha_corte <= fechaCorteMesActual).OrderBy(c => c.numero_cuota).ToList();
    var tasaMoraCredito = Connection.ContextC10.tbl_tasas_mora.FirstOrDefault(t => t.fk_id_tipo_credito == credito.fk_id_tipo_credito &&
                                                                                   t.fecha_inicio <= DateTime.Today && t.fecha_fin >= DateTime.Today && t.estado);
    if (tasaMoraCredito == null)
    {
        tasaMoraCredito = Connection.ContextC10.tbl_tasas_mora.Where(t => t.fk_id_tipo_credito == credito.fk_id_tipo_credito &&
                                                                          t.estado).OrderByDescending(t => t.fecha_fin).First();
    }

    foreach (tbl_programacion_cuotas cuota in cuotas)...
}
```

En el mismo método con la lista de créditos encontrada anteriormente se inicia un ciclo recorriendo la lista en el orden especificado, en un primer momento se llama un procedimiento llamado *sp_validar_vigencia_credito* y este es llamado en cada crédito que, aunque se explicara con más detalle el procedimiento almacenado más adelante en resumen este validara si el crédito evaluado está bien amortizado o si este ya se debería encontrar cancelado por el pago completo de su capital. El procedimiento almacenado devolverá si el crédito analizado ya debería estar en estado cancelado y si en el pago total debe haber una devolución de saldo para que allá coherencia en el cálculo. Esto se hizo debido a que a veces se encontraba una alteración en los datos y un crédito se encontraba vigente a pesar de que el cliente pago más capital del debido y esto hacía que el pago total tuviera en ocasiones valores positivos.

Después de validar el estado del crédito, obtengo la tasa de interés que se encuentra actualmente en vigencia, ya que más adelante será utilizado para saber el impuesto del IVA que se encuentra en vigencia. Con respecto al valor propio de la tasa de interés se utiliza el valor con el que se creó el crédito.

Otros valores que se buscan son si dentro de la lista de créditos se encuentra un crédito refinanciado, la tasa de mora vigente a la fecha y las cuotas con las que se guarda la amortización correspondiente. Con respecto a estas cuotas se inicia un nuevo ciclo recorriendo cuota por cuota en orden ascendente para ver su estado y los saldos que adeuda en cada cuota.



```
foreach (tbl_programacion_cuotas cuota in cuotas)
{
    if (cuota.pago_total != null)
    {
        if (cuota.pago_total == false) // Caso en el que se a hecho un pagoParcial
        {
            var pendientes = Connection.ContextC10.sp_pagos_pendientes(cuota.id_programacioncuotas).FirstOrDefault();
            var abonoRealizado = Connection.ContextC10.sp_consulta_abono_cuota(cuota.id_programacioncuotas).FirstOrDefault();
            var pagoAnticipado = Connection.ContextC10.sp_consulta_pagos_anticipados(cuota.id_programacioncuotas,
                                                    fechaCorteMesActual.AddMonths(-1), fechaCorteMesActual).FirstOrDefault();

            //Consulta el saldo pendiente de cada valor con un sp
            var IvaMoraPendiente = (decimal)pendientes.IvaMoraPendiente;
            var MoraPendiente = (decimal)pendientes.MoraPendiente;
            var IvaAvalPendiente = (decimal)pendientes.IvaAvalPendiente;
            var AvalPendiente = (decimal)pendientes.AvalPendiente;
            var IvaInteresPendiente = (decimal)pendientes.IvaInteresPendiente;
            var InteresPendiente = (decimal)pendientes.InteresPendiente;
            var CapitalPendiente = (decimal)pendientes.CapitalPendiente;

            // ==2
            if (Refinanciado && cuota.numero_cuota == Constant.ParametrosPagos.CuotaUno && cuota.fecha_corte > fechaActual)
            {
                pagoRef.ValorCuotaCreditoRefinanciado = CapitalPendiente + InteresPendiente + IvaInteresPendiente;
                pagoRef.PrimerPagoRefinanciacion = true;
            }
            else...
        }
    }
    else...
}
```

En el ciclo que recorre cuota a cuota del crédito, primero se empieza evaluando en qué estado se encuentra. Si el estado es diferente de nulo significa que ya esta cuota se puede encontrar en dos estados, el valor será 1 (true) si la cuota fue pagada por completo y este estado será ignorado por completo en el cálculo o tendrá un valor de 0 (false) si la cuota tiene un abono realizado a ella y no ha sido saldado por completo, este último estado solo podrá darse en el caso en que la cuota fue presupuestada en el proceso de corte.

Si la cuota está en estado false, se llaman 3 procedimiento almacenados que traerán diferente información del estado de la cuota, el primero *sp_pagos_pendientes* al enviarle el id de la cuota este devolverá los valores pendientes por pagar de cada ítem. El segundo es *sp_consulta_abono_cuota* que traerá la información del último abono realizado como la fecha y el detalle de cómo se dividió el pago y el tercero es *sp_consulta_pagos_anticipados* que devolver el valor de la mora e IVA mora que pudo haber pagado de manera anticipada en el último abono.

Teniendo estos datos ya guardados, se realiza una condición para determinar si la cuota pertenece a un crédito refinanciado, si esta es la primera cuota y si no ha pasado por su fecha de corte. Si cumple con estas condiciones en el objeto de pagoRef que será devuelto en el método se establece su status (*PrimerPagoRefinanciacion*) en true y el valor pendiente que tenga actualmente la cuota. Esta condición es debido a que la primera cuota de refinanciación se debe pagar por completo, tal cual como se programó sin importar si ha pasado o no por presupuesto. Una cuota de un crédito normal funciona de manera diferente.

```
else
{
    if (abonoRealizado != null)
    {
        if (abonoRealizado.fecha < fechaPagoMesActual)//Si el pago se hizo en el plazo del pago
        {
            if (fechaActual <= fechaPagoMesActual)
            {
                PAGO_MINIMO += CapitalPendiente + InteresPendiente + IvaInteresPendiente + AvalPendiente + IvaAvalPendiente + MoraPendiente + IvaMoraPendiente;
                PAGO_TOTAL += InteresPendiente + IvaInteresPendiente + AvalPendiente + IvaAvalPendiente + MoraPendiente + IvaMoraPendiente;
            }
            else
            {
                //CHECK
                var diasMora = fechaActual.Subtract(fechaPagoMesActual).Days;
                var tasaMora = tasaMoraCredito.valor_efectivo_mensual / Constant.ParametrosPagos.Porciento;
                var moraAnticipada = Math.Round((((CapitalPendiente + AvalPendiente + IvaAvalPendiente) * tasaMora) / Constant.ParametrosPagos.DiasMes) * diasMora, 0, MidpointRounding.AwayFromZero);
                var ivaMoraAnticipada = Math.Round(moraAnticipada * (tasaMoraCredito.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);

                PAGO_MINIMO += CapitalPendiente + InteresPendiente + IvaInteresPendiente + AvalPendiente + IvaAvalPendiente + MoraPendiente + IvaMoraPendiente + moraAnticipada + ivaMoraAnticipada;
                PAGO_TOTAL += InteresPendiente + IvaInteresPendiente + AvalPendiente + IvaAvalPendiente + MoraPendiente + IvaMoraPendiente + moraAnticipada + ivaMoraAnticipada;
            }
        }
        else...
    }
    else
    {
        var diasMora = fechaActual.Subtract(fechaPagoMesActual).Days;
        var tasaMora = tasaMoraCredito.valor_efectivo_mensual / Constant.ParametrosPagos.Porciento;
        var moraAnticipada = Math.Round((((CapitalPendiente + AvalPendiente + IvaAvalPendiente) * tasaMora) / Constant.ParametrosPagos.DiasMes) * diasMora, 0, MidpointRounding.AwayFromZero);
        var ivaMoraAnticipada = Math.Round(moraAnticipada * (tasaMoraCredito.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);

        PAGO_MINIMO += CapitalPendiente + InteresPendiente + IvaInteresPendiente + AvalPendiente + IvaAvalPendiente + MoraPendiente + IvaMoraPendiente + moraAnticipada + ivaMoraAnticipada;
        PAGO_TOTAL += InteresPendiente + IvaInteresPendiente + AvalPendiente + IvaAvalPendiente + MoraPendiente + IvaMoraPendiente + moraAnticipada + ivaMoraAnticipada;
    }
}
```

Al entrar en la condición del *else*, se evalúa si la variable *abonoRealizado* que retorno el procedimiento almacenado es diferente de nulo, si se cumple esto es porque se tiene información con respecto al abono que puso a la cuota en ese estado y se realiza una serie de validaciones adicionales con respecto al abono en cuestión, que se puede resumir en sí a la fecha actual los saldos pendientes de la cuota son los mismos o si se debe calcular mora anticipada adicional a la cuota con respecto a la fecha del mismo abono. Si la variable abonoRealizado es igual a *null* se calculará una mora anticipada. Al final a las variables de *PAGO_MINIMO* se le ira acumulando los valores de capital, interés, aval, mora e IVA pendientes de la cuota y al *PAGO_TOTAL* se le acumulara los ítems de interés, aval, mora e IVA descartando en este caso el capital que ya fue considerado desde un principio.

```
else //Caso en que no se a aplicado ningun pago a la cuota
{
    //Antes de corte
    if (cuota.fecha_corte >= fechaPagoMesActual)
    {
        var dias = 0;
        if (cupo.dia_corte == Constant.ParametrosPagos.DiaCorte15)
        {
            dias = fechaActual.Subtract(fechaCorteMesActual.AddMonths(-1)).Days;
        }
        else
        {
            dias = fechaActual.Day;
        }
        if (cuota.numero_cuota == Constant.ParametrosPagos.CuotaUno)
        {
            if (Refinanciado)
            {
                pagoRef.ValorCuotaCreditoRefinanciado = cuota.valor_capital + cuota.valor_interes + cuota.valor_iva_interes;
                pagoRef.PrimerPagoRefinanciacion = true;
            }
            var monto = credito.monto_desembolsado - credito.monto_capital_pagado;
            var dias_ = fechaActual.Subtract(credito.fecha_desembolso.Date).Days;
            var tasa = credito.tasa_interes / Constant.ParametrosPagos.Porciento;

            var Intereses = Math.Round((decimal)((monto * tasa) / Constant.ParametrosPagos.DiasMes) * dias_, 0, MidpointRounding.AwayFromZero); ;
            var IvaInteres = Math.Round(Intereses * (tasaInteres.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);
            PAGO_TOTAL += Intereses + IvaInteres;
        }
        else
        {
            var Intereses = Math.Round(((cuota.valor_interes) / Constant.ParametrosPagos.DiasMes) * dias, 0, MidpointRounding.AwayFromZero);
            var IvaInteres = Math.Round(Intereses * (tasaInteres.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);
            PAGO_TOTAL += Intereses + IvaInteres;
        }
        PAGO_CUOTA_SIGUIENTE += cuota.valor_capital + cuota.valor_interes + cuota.valor_iva_interes + cuota.valor_aval + cuota.valor_iva_aval;
    }
    else...
}
```

Evaluando ya el caso en que una cuota tenga un abono al tener el estado del *pago_total* en 0, que en teoría solo debería haber una cuota por crédito con esta condición. Se pasa a evaluar las cuotas que no tienen ningún pago en la cuota, estas cuotas se separan por la condición de si ya paso por corte o no.

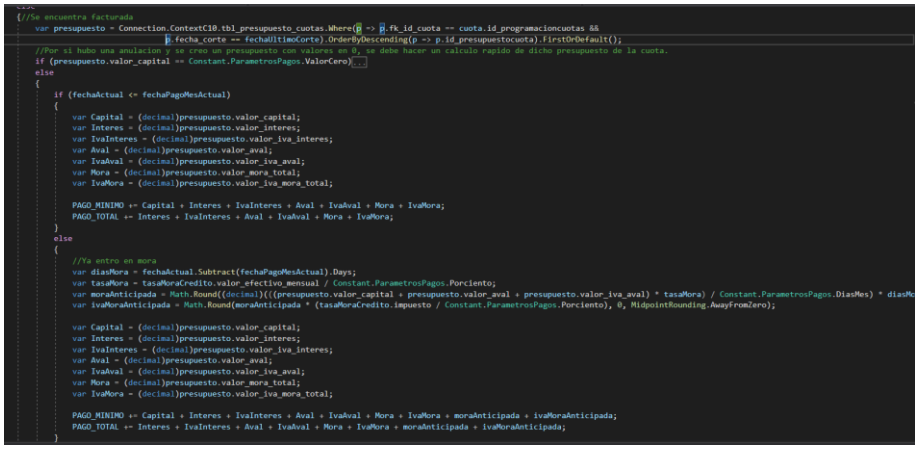
Si la cuota se encuentra antes de corte será el caso en que se deba calcular unos intereses anticipados del crédito y también el valor que se debe retornar sobre el pago de la cuota siguiente. Para calcular los intereses anticipados primero se calcula el número de días que tiene el cliente dependiendo de su día de corte. Luego se evalúa si la cuota que se está evaluando es la primera del crédito, si lo es, se valida que el crédito es de tipo refinanciacion ya que si lo es se deberá crear el objeto a retornar de pagoRef con el status en true y el valor completo de la cuota a pagar; si no es un crédito refinanciado al ser la primera cuota y esta encontrarse antes de corte el cálculo de días será diferente por lo que se reasigna este valor calculando los días transcurridos desde la fecha de desembolso.

Para calcular los intereses anticipados de la primera cuota se toma el saldo completo del crédito y se multiplica por la tasa con que se sacó el crédito, dividido en 30 días y multiplicado por el número de días calculado anteriormente.

Si el número de cuota es diferente a la primera se mantiene el cálculo de días del principio y los intereses anticipados se calculan con respecto al valor programado de la cuota del interés, dividiéndolo por 30 días y multiplicándolo luego por el número de días.

Con el cálculo realizado de los intereses anticipados estos se incluirán solamente en el PAGO_TOTAL junto al impuesto del valor calculado.

Para el valor de PAGO_CUOTA_SIGUIENTE simplemente se sumará por completo los valores programados de la cuota, el cual ira acumulando este valor.



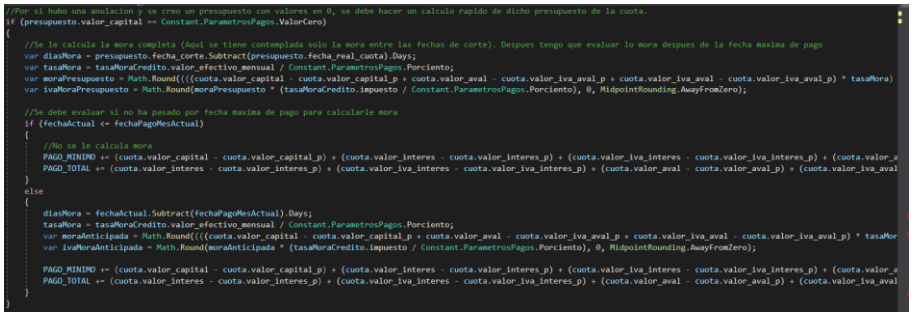
```
{//Se encuentra facturada
    var presupuesto = Connection.ContextC10.tbl_presupuesto_cuotas.Where(p => p.fk_id_cuota == cuota.id_programacioncuotas &&
                                                                          p.fecha_corte == fechaUltimoCorte).OrderByDescending(p => p.id_presupuestocuota).FirstOrDefault();
    //Por si hubo una anulacion y se creo un presupuesto con valores en 0, se debe hacer un calculo rapido de dicho presupuesto de la cuota.
    if (presupuesto.valor_capital == Constant.ParametrosPagos.ValorCero)...
    else
    {
        if (fechaActual <= fechaPagoMesActual)
        {
            var Capital = (decimal)presupuesto.valor_capital;
            var Interes = (decimal)presupuesto.valor_interes;
            var IvaInteres = (decimal)presupuesto.valor_iva_interes;
            var Aval = (decimal)presupuesto.valor_aval;
            var IvaAval = (decimal)presupuesto.valor_iva_aval;
            var Mora = (decimal)presupuesto.valor_mora_total;
            var IvaMora = (decimal)presupuesto.valor_iva_mora_total;

            PAGO_MINIMO += Capital + Interes + IvaInteres + Aval + IvaAval + Mora + IvaMora;
            PAGO_TOTAL += Interes + IvaInteres + Aval + IvaAval + Mora + IvaMora;
        }
        else
        {
            //Ya entro en mora
            var diasMora = fechaActual.Subtract(fechaPagoMesActual).Days;
            var tasaMora = tasaMoraCredito.valor_efectivo_mensual / Constant.ParametrosPagos.Porciento;
            var moraAnticipada = Math.Round((decimal)(((presupuesto.valor_capital + presupuesto.valor_aval + presupuesto.valor_iva_aval) * tasaMora) / Constant.ParametrosPagos.DiasMes) * diasMo
            var ivaMoraAnticipada = Math.Round(moraAnticipada * (tasaMoraCredito.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);

            var Capital = (decimal)presupuesto.valor_capital;
            var Interes = (decimal)presupuesto.valor_interes;
            var IvaInteres = (decimal)presupuesto.valor_iva_interes;
            var Aval = (decimal)presupuesto.valor_aval;
            var IvaAval = (decimal)presupuesto.valor_iva_aval;
            var Mora = (decimal)presupuesto.valor_mora_total;
            var IvaMora = (decimal)presupuesto.valor_iva_mora_total;

            PAGO_MINIMO += Capital + Interes + IvaInteres + Aval + IvaAval + Mora + IvaMora + moraAnticipada + ivaMoraAnticipada;
            PAGO_TOTAL += Interes + IvaInteres + Aval + IvaAval + Mora + IvaMora + moraAnticipada + ivaMoraAnticipada;
        }
```

Si la cuota no se encuentra antes de corte, significa que ya debe haber un presupuesto creado por esa cuota, por lo tanto, se busca dicho presupuesto y se hace valida nuevamente si la fecha actual de consulta es menor o igual a la fecha máxima de pago que tiene el cliente, si esto es cierto en el PAGO_MINIMO se sumaran todos los valores presupuestados de la cuota (capital, interés, aval, mora e IVA) y para el PAGO_TOTAL se sumaran solo los valores de interés, aval, mora e IVA. Si la fecha actual es mayor a la fecha máxima del pago significa que el cliente incurrió en mora y se procede a calcular los días que han transcurrido desde la fecha máxima que tenía, se busca la tasa de mora vigente y se calcula la mora anticipada utilizando la formula 3 y en este caso al PAGO_MINIMO y al PAGO_TOTAL se le suman los mismos items que el caso anterior, pero se le adiciona el valor de mora anticipada calculado anteriormente.



```
//Por si hubo una anulacion y se creo un presupuesto con valores en 0, se debe hacer un calculo rapido de dicho presupuesto de la cuota.
if (presupuesto.valor_capital == Constant.ParametrosPagos.ValorCero)
{
    //Se le calcula la mora completa (Aqui se tiene contemplado solo la mora entre las fechas de corte). Despues tengo que evaluar lo mora despues de la fecha maxima de pago
    var diasMora = presupuesto.fecha_corte.Subtract(presupuesto.fecha_real_cuota).Days;
    var tasaMora = tasaMoraCredito.valor_efectivo_mensual / Constant.ParametrosPagos.Porciento;
    var moraPresupuesto = Math.Round((((cuota.valor_capital - cuota.valor_capital_p + cuota.valor_aval - cuota.valor_iva_aval_p + cuota.valor_iva_aval - cuota.valor_iva_aval_p) * tasaMora)
    var ivaMoraPresupuesto = Math.Round(moraPresupuesto * (tasaMoraCredito.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);

    //Se debe evaluar si no ha pasado por fecha maxima de pago para calcularle mora
    if (fechaActual <= fechaPagoMesActual)
    {
        //No se le calcula mora
        PAGO_MINIMO += (cuota.valor_capital - cuota.valor_capital_p) + (cuota.valor_interes - cuota.valor_interes_p) + (cuota.valor_iva_interes - cuota.valor_iva_interes_p) + (cuota.valor_a
        PAGO_TOTAL += (cuota.valor_interes - cuota.valor_interes_p) + (cuota.valor_iva_interes - cuota.valor_iva_interes_p) + (cuota.valor_aval - cuota.valor_aval_p) + (cuota.valor_iva_aval
    }
    else
    {
        diasMora = fechaActual.Subtract(fechaPagoMesActual).Days;
        tasaMora = tasaMoraCredito.valor_efectivo_mensual / Constant.ParametrosPagos.Porciento;
        var moraAnticipada = Math.Round((((cuota.valor_capital - cuota.valor_capital_p + cuota.valor_aval - cuota.valor_iva_aval_p + cuota.valor_iva_aval - cuota.valor_iva_aval_p) * tasaMor
        var ivaMoraAnticipada = Math.Round(moraAnticipada * (tasaMoraCredito.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);

        PAGO_MINIMO += (cuota.valor_capital - cuota.valor_capital_p) + (cuota.valor_interes - cuota.valor_interes_p) + (cuota.valor_iva_interes - cuota.valor_iva_interes_p) + (cuota.valor_a
        PAGO_TOTAL += (cuota.valor_interes - cuota.valor_interes_p) + (cuota.valor_iva_interes - cuota.valor_iva_interes_p) + (cuota.valor_aval - cuota.valor_aval_p) + (cuota.valor_iva_aval
    }
}
```

Hay una condición extra y es si se detecta que el presupuesto viene con un valor en cero (0), si esto ocurre es muy probable que al cliente le hayan anulado un pago y se tuvo que crear un presupuesto en 0 para conocer dichos casos. Cuando sucede eso se hará la misma lógica descrita anteriormente por si el cliente incurrió o no en mora anticipada pero ya los valores no se tendrán en cuenta del presupuesto si no de la programación que tenga la cuota en su momento.

En este punto, ciclo tras ciclo tanto en las cuotas como en los créditos de cada cliente se ira acumulando los valores en las variables de PAGO_MINIMO y PAGO_TOTAL hasta que finalizado el ciclo se tenga el valor que el cliente adeuda y la información adicional que retorna el método explicado.

Formulas utilizadas

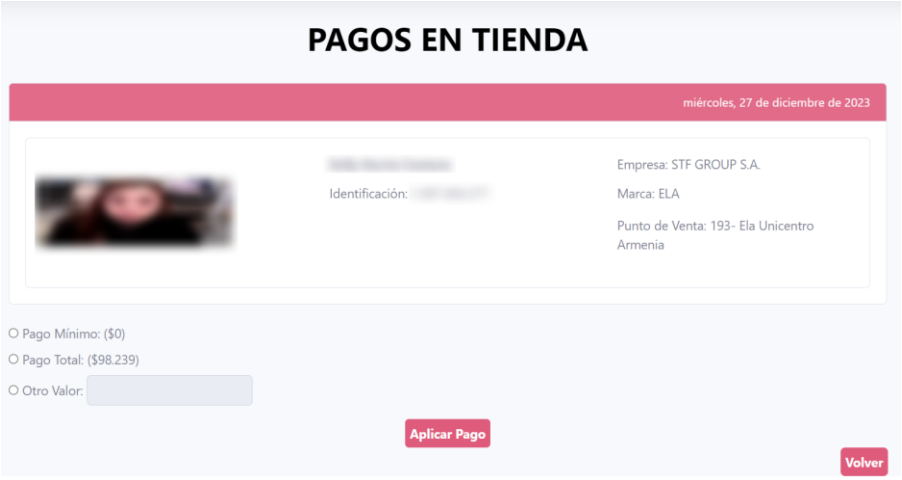
$$InteresAnt = \left(\frac{(saldoCredito \cdot tasaInteres)}{30}\right) \cdot dias$$

1

$$InteresAnt = \left(\frac{cuotaValorInteres}{30}\right) \cdot dias$$ 2

$$MoraAnt = \left(\frac{\left(\left((Capital + Aval + IvaAval)\right)\cdot tasaMora\right)}{30}\right) \cdot dias \qquad 3$$

Cuando se tenga el cálculo hecho y la información adicional recopilada en el controlador de Create se devolverá la vista el cual se vera de la siguiente manera.



Así se vería la vista con la información del pago mínimo y total que tenga en el dia de la consulta, con información adicional de la foto, nombre del cliente y el punto de venta donde se haría el recaudo.

```
[HttpPost]
[ValidateAntiForgeryToken]
0 referencias
public ActionResult Create(InformacionPagoDTO infoPago, string ValorPago)  //InformacionPagoDTO payment
{
    var result = new OperationResponse();

    try
    {
        var saldo = Convert.ToDecimal(ValorPago);

        if (!ValidateSession(true))...
        else
        {
            if (saldo < 1 || saldo > infoPago.PagoTotal + 1)
            {
                throw new CustomException(ParameterManager.GetParamsms(Constant.MensajesKey.ValorNoPermitido).Value);
            }
            (bool permiso, string mensaje) = MPayment.ConsultaDetalladaLimiteRecaudo(infoPago.IdOficina, saldo);
            if (!permiso)
            {
                throw new CustomException(mensaje);
            }
            if (MPayment.HuboMovimientoDelCupoDespuesDeConsulta(infoPago.FechaConsulta, infoPago.Cliente.Id))
            {
                throw new CustomException(ParameterManager.GetParamsms(Constant.MensajesKey.CambioDePagoDesdeLaConsulta).Value);
            }
            result = MPayment.MaestroAplicarPagoTienda(infoPago, saldo, ControllerContext);
        }
    }
    catch (Exception ex)
    {
        result.Message = ex.Message;
        result.Completed = false;
        result.IsError = true;
        result.Status = false;
    }
    Security.ControlAuditoria(SessionUser.UserName, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.CrearPago, result.Message, new {info = infoPago, totalPago = ValorPago });
    return Json(result);
}
```

Cuando se da en el botón de aplicar el pago llama por método POST al endpoint Create que recibe como parámetro un objeto con la información del pago que se mostró en la vista de donde viene el llamado y el valor del pago correspondiente que quiere realizar.

En un primer momento se valida la sesión para asegurar que esta no haya caducado y poder registrar el usuario que aplico el pago, si la sesión sigue vigente se valida que el saldo a pagar que se ingreso sea mayor a un peso y sea menor que el valor total que tenga la clienta y fue mostrado en pantalla. Esto para evitar que en tiendas se recaude más de lo debido y los clientes empiecen a tener saldos a favor en sus créditos.

Si pasa el primer filtro, se evalúa llamando al método *ConsultaDetalladaLimiteRecaudo()* explicado más adelante para saber si la oficina puede recaudar el valor ingresado que recibió el método, si no puede se crea una excepción controlada para retornar a la vista un mensaje parametrizado de límite de recaudo de la empresa a la que pertenece el punto de venta, si no, continua con la próxima validación. En este punto se llama al método *HuboMovimientoDeCupoDespuesDeConsulta()*, el cual me devolver un true si el método (explicado más adelante) detecta que el cliente tuvo un movimiento que afecto el pago que pretende hacer, obligando así una nueva consulta por parte del usuario, si no es así ya se llama al maestro de aplicar el pago que será la clase intermediaria con la clase *PagosService* para gestionar el proceso.

```
1 referencia
public OperationResponse MaestroAplicarPagoTienda(InformacionPagoDTO infoPago, decimal saldo, ControllerContext context)
{
    var result = new OperationResponse();
    try
    {
        var pagoDTO = new PagoDTO()
        {
            ClienteDTO = infoPago.Cliente,
            Cliente = infoPago.Cliente.NombreCompleto,
            Cedula = infoPago.NumeroIdentificacion,
            Celular = infoPago.Cliente.Celular,
            Correo = infoPago.Cliente.Correo,
            IdCliente = infoPago.Cliente.Id,
            IdOficina = infoPago.IdOficina,
            Oficina = infoPago.Marca + " " + infoPago.Oficina,
            Empresa = infoPago.Empresa,
            IdEmpresa = infoPago.IdEmpresa,
            Nit = infoPago.Nit,
            Date = DateTime.Now,
            ValorPago = saldo,
            PagoMinimo = infoPago.PagoMinimo,
            PagoTotal = infoPago.PagoTotal,
            TipoPago = Constant.TipoPagoComprobante.Fisico,
            Asesor = infoPago.Asesor,
            Telefono = infoPago.Telefono,
            Ciudad = infoPago.Ciudad,
            Direccion = infoPago.Direccion
        };

        result = Pagos.AplicarPago(pagoDTO, context, true);
    }
    catch
    {
        result.Completed = false;
        result.Status = false;
        result.Message = "Hubo un error intentando aplicar el pago, intenta nuevamente.";
    }
    return result;
}
```

El método *MaestroAplicarPagoEnTienda()* es solo un intermediario entre el controlador web que recibe el pago y la clase PagosService que es donde se aloja la lógica fuerte para consultar y aplicar los pagos. Este método recibe como parámetros los mismos objetos recibidos por el controlador Create y el parámetro ControllerContext que es usado y necesitado para poder crear el ticket correspondiente del pago. Con dichos datos se crea el objeto pagoDTO que contiene todos los valores necesarios para que PagosService procese y aplique correctamente el pago, pues en este punto es donde será llamado el método *AplicarPago()* de PagosService.

```
#region Pagos
7 referencias
public OperationResponse AplicarPago(PagoDTO payment, ControllerContext context = null, bool pagoEnTienda = false)
{
    var result = new OperationResponse();
    var ticket = new Ticket();

    //creamos el ámbito de la transacción
    using (var dbContextTransaction = Connection.ContextC10.Database.BeginTransaction())...
    if (result.Completed)
    {
        var pathTicket = PrintTicket.CrearTicketPago(ticket, context);
        if (pagoEnTienda)
        {
            result.Data = pathTicket.Data;
            //Envio SMS
            Notification.EnviarSMSPago(payment.ValorPago, payment.Celular);
            //Envio del Email
            Notification.EnviarCorreoPago(payment.Correo, payment.ValorPago, pathTicket.Data, ticket);
            //Envio Whatsap
            var encendidoWhatsapp = bool.Parse(ParameterManager.GetParamNow(Constant.ParamKey.EncendidoWhatsappPago).Value);
            if (encendidoWhatsapp)
                Notification.EnviarWhatsappPago(payment.ValorPago, payment.Celular);
        }
    }

    return result;
}
```

El método de AplicarPago() será en general el que se use para aplicar los pagos normales por tienda, consignación o cargue de consignaciones. En los pagos online varia un poco en la forma en que se usa el control de transacciones por lo que quedo en un método adicional, de igual manera los métodos usados internamente si serán iguales.

En el método en general se inicia una transacción para asegurar de que si llegase haber un error interno en algún punto se restableciera todos los cambios realizados en base de datos en el movimiento que se hace al aplicar el pago, si todo el proceso sale correcto en la transacción se guarda y se realiza commit de la transacción y se inicia el proceso de crear el ticket informativo del pago y posterior notificación al cliente, la notificación y creación del ticket depende de un parámetro recibido en el método llamado *pagoEnTienda* que tendrá un valor por defecto en false para cuando no se envié este proceso de notificación y creación de ticket se omita. Los métodos usados y explicados más adelante son los encargados de crear el ticket, notificar por SMS y correo y si esta activado la notificación por WhatsApp que en el momento al crear este documento no está activado por falta de una plantilla de Meta.

Dentro de la transacción, se empieza a crear el objeto con el que se creara el ticket del pago, incluyendo datos del cliente y sitio del pago, y se identificara que tipo de documento se debe usar para así mismo obtener el consecutivo

```
if (payment.TipoPago.Equals(Constant.TipoPagoComprobante.Refinaciacion))
{
    consecutivo = Connection.ContextC10.tbl_consecutivos.FirstOrDefault(c => (c.fk_id_oficina == payment.IdOficina)
    && c.fk_id_documentos_transaccionales == documentoClasificacion.fk_id_doc_nota_credito_refinanciacion
    && c.vigencia_fecha_inicio <= DateTime.Today && c.vigencia_fecha_fin >= DateTime.Today);
    documento = documentoClasificacion.fk_id_doc_nota_credito_refinanciacion;
}
else
{
    consecutivo = Connection.ContextC10.tbl_consecutivos.FirstOrDefault(c => (c.fk_id_oficina == payment.IdOficina)
    && c.fk_id_documentos_transaccionales == documentoClasificacion.fk_id_doc_recibo_caja
    && c.vigencia_fecha_inicio <= DateTime.Today && c.vigencia_fecha_fin >= DateTime.Today);
    documento = documentoClasificacion.fk_id_doc_recibo_caja;
}
```

Puede haber dos tipos de pagos, pago por refinanciacion el cual se usa un documento clasificado como NFR, y un documento llamado recibo de caja que serán los recaudos normales de los clientes. Al saber el tipo de documento a usar gracias al objeto recibido en el método, este obtendrá el número del consecutivo actual que cumpla con la vigencia establecida con el que deberá crearse el documento a continuación.

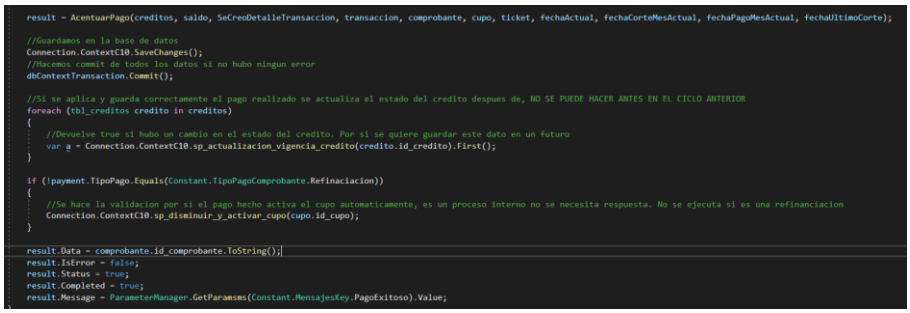
```
var comprobante = new tbl_comprobantes()
{
    fk_id_documento_transaccional = documento,
    consecutivo = consecutivo.actual + 1,
    fecha = DateTime.Now,
    fk_id_oficina = payment.IdOficina,
    fk_id_empresa = payment.IdEmpresa,
    fk_id_cliente = payment.IdCliente,
    fk_id_cupo = cupo.id_cupo,
    valor_total = payment.ValorPago,
    tipo_pago = payment.TipoPago,
    fecha_consignacion = payment?.FechaConsignacion,
    tipo_transaccion = payment?.TipoTransaccion,
    banco_consignacion = payment?.Banco,
    observaciones = payment?.Observaciones,
    fk_id_asesor = asesor?.id_asesor,
    anulado = false
};
Connection.ContextC10.tbl_comprobantes.Add(comprobante);
var transaccion = new tbl_control_transacciones()
{
    fk_id_cupo = cupo.id_cupo,
    fecha = DateTime.Now.Date,
    hora = DateTime.Now.TimeOfDay,
    valor_actual = payment.PagoMinimo,
    valor_total = payment.PagoTotal,
    tbl_comprobantes = comprobante
};
Connection.ContextC10.tbl_control_transacciones.Add(transaccion);

//SUMAR AL CONSECUTIVO
consecutivo.actual += 1;

ticket.consecutivo = comprobante.fk_id_documento_transaccional + comprobante.consecutivo.ToString();
```

Con los datos necesarios se creará ya el comprobante del pago como documento y así mismo la transacción asociada a este documento, después de esto se sumará

el consecutivo el valor de una unidad para el próximo pago de la tienda y llevar una trazabilidad única.



```
result = AcentuarPago(creditos, saldo, SeCreoDetalleTransaccion, transaccion, comprobante, cupo, ticket, fechaActual, fechaCorteMesActual, fechaPagoMesActual, fechaUltimoCorte);

//Guardamos en la base de datos
Connection.ContextC10.SaveChanges();
//Hacemos commit de todos los datos si no hubo ningun error
dbContextTransaction.Commit();

//Si se aplica y guarda correctamente el pago realizado se actualiza el estado del credito despues de, NO SE PUEDE HACER ANTES EN EL CICLO ANTERIOR
foreach (tbl_creditos credito in creditos)
{
    //Devuelve true si hubo un cambio en el estado del credito. Por si se quiere guardar este dato en un futuro
    var a = Connection.ContextC10.sp_actualizacion_vigencia_credito(credito.id_credito).First();
}

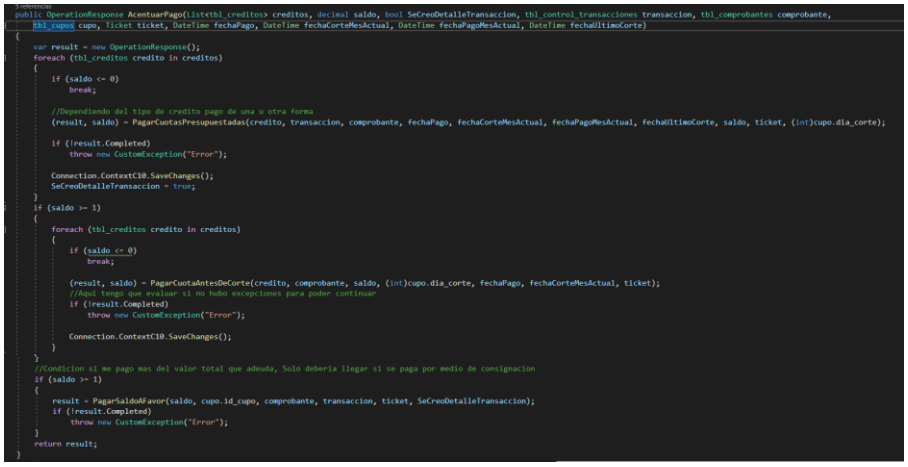
if (!payment.TipoPago.Equals(Constant.TipoPagoComprobante.Refinaciacion))
{
    //Se hace la validacion por si el pago hecho activa el cupo automaticamente, es un proceso interno no se necesita respuesta. No se ejecuta si es una refinanciacion
    Connection.ContextC10.sp_disminuir_y_activar_cupo(cupo.id_cupo);
}

result.Data = comprobante.id_comprobante.ToString();
result.IsError = false;
result.Status = true;
result.Completed = true;
result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.PagoExitoso).Value;
```

Después de crear el comprobante ya se hace el llamado del método *AcentuarPago()* el cual hará ya todo el proceso correspondiente para quedar el pago asentado en las cuotas de los créditos y realizar el detalle del ticket con los movimientos que se hagan allí. Si el resultado de este es exitoso guardará los cambios y de inmediato hará el commit de la transacción para desbloquear las tablas y poder continuar con el proceso.

Al finalizar el commit se realizan dos procesos, el primero es un proceso que se hace crédito por crédito para validar si estos ya han sido pagos y cambiar el estado de estos a cancelados, si el crédito sigue con saldo mantendrá su estado normal.

El otro proceso ya será un llamado por cupo, solo si el tipo de pago es diferente de refinanciacion, se llamará un procedimiento almacenado que validará el estado del cupo en el que se encuentre actualmente el cliente y si se encuentra en un estado determinado al estar en una empresa de cobranza, y el pago realizado saldo por completo la deuda del cliente, este desbloqueará el cupo automáticamente y le disminuirá a la mitad el monto aprobado. El procedimiento almacenado será explicado más adelante con más detalle.



```
5 references
public OperationResponse AcentuarPago(List<tbl_creditos> creditos, decimal saldo, bool SeCreoDetalleTransaccion, tbl_control_transacciones transaccion, tbl_comprobantes comprobante,
    tbl_cupos cupo, Ticket ticket, DateTime fechaPago, DateTime fechaCorteMesActual, DateTime fechaPagoMesActual, DateTime fechaUltimoCorte)
{
    var result = new OperationResponse();
    foreach (tbl_creditos credito in creditos)
    {
        if (saldo <= 0)
            break;

        //Dependiendo del tipo de credito pago de una u otra forma
        (result, saldo) = PagarCuotasPresupuestadas(credito, transaccion, comprobante, fechaPago, fechaCorteMesActual, fechaPagoMesActual, fechaUltimoCorte, saldo, ticket, (int)cupo.dia_corte);

        if (!result.Completed)
            throw new CustomException("Error");

        Connection.ContextC10.SaveChanges();
        SeCreoDetalleTransaccion = true;
    }
    if (saldo >= 1)
    {
        foreach (tbl_creditos credito in creditos)
        {
            if (saldo <= 0)
                break;

            (result, saldo) = PagarCuotaAntesDeCorte(credito, comprobante, saldo, (int)cupo.dia_corte, fechaPago, fechaCorteMesActual, ticket);
            //Aqui tengo que evaluar si no hubo excepciones para poder continuar
            if (!result.Completed)
                throw new CustomException("Error");

            Connection.ContextC10.SaveChanges();
        }
    }
    //Condicion si me pago mas del valor total que adeuda, Solo deberia llegar si se paga por medio de consignacion
    if (saldo >= 1)
    {
        result = PagarSaldoAFavor(saldo, cupo.id_cupo, comprobante, transaccion, ticket, SeCreoDetalleTransaccion);
        if (!result.Completed)
            throw new CustomException("Error");
    }
    return result;
}
```

El método de AcentuarPago() será encargado de separar 3 procesos generales en el pago, que son

1. Pagar cuotas presupuestadas: Son las que ya han pasado por el proceso de corte. Lo que en pantalla se muestra como pago mínimo.
2. Pagar cuotas antes de corte: El segundo proceso depende si el saldo del pago es mayor a 1 en este punto, si es asi, se procede a aplicar un abono sobre las cuotas no presupuestadas que son las que no han pasado por corte.
3. Pagar saldo a favor: El tercer proceso depende si el saldo del pago sigue siendo mayor a 1 en este punto, que será el momento en que con el pago ingresado supero su pago total y se debe aplicar un saldo a favor en el crédito para llevar control de esos pagos extras. Este último punto solo podría suceder en las consignaciones que hacen uso de este método pues allí no está el control de no poder ingresar un valor mayor que el total. En lo que respecta a pago en tienda, en este punto no debería llegar.

Cada proceso será llamado por cada crédito que tenga vigente el cliente. Después de terminar un proceso por cada crédito es que se pasa a evaluar si el saldo del pago da para continuar con el segundo proceso.

```
var liquidacion = new CreditoPagoDTO()
{
    IdCredito = credito.id_credito,
    FechaDesembolso = credito.fecha_desembolso,
    MontoCapitalPagado = (decimal)credito.monto_capital_pagado,
    Vigencia = credito.vigente,
    EstadoCredito = credito.fk_id_estado,
    Cuotas = CustomParser.ParseList(cuotas)
};

var detalleTransaccion = new tbl_control_transacciones_detalle()
{
    tbl_control_transacciones = transaccion,
    tbl_creditos = credito,
    periodo_gracia = false,
    liquidacion = JsonConvert.SerializeObject(liquidacion) // Json del objeto que guardara las cuotas de cada credito
};
Connection.ContextC10.tbl_control_transacciones_detalle.Add(detalleTransaccion);


//las cuotas que ya hann pasado por corte se filtraran para saber que valores ya estan presupuestados
var cuotasPresupuestadas = cuotas.Where(c => c.fecha_corte < fechaCorteMesActual).ToList();
```

En el método *PagarCuotasPresupuestadas()* que realiza el pago de lo ya mencionado, se encuentra el momento donde se crea el detalle del control de la transacción, para ello se crea un objeto llamado liquidación que guardara información actual del estado del crédito y como se encuentran las cuotas antes de empezar aplicar el pago, es como una foto del estado en que se encuentra el crédito.

```
if (cuotasPresupuestadas.Count != 0)
{
    var detalleTicket = new DetalleTicket();
    detalleTicket.credito = credito.id_credito;

    foreach (tbl_programacion_cuotas cuota in cuotasPresupuestadas)...
    ticket.detallePago.Add(detalleTicket);
}
else
{
    if (Refinanciado)
    {
        var response = new OperationResponse();

        (response, saldo) = PagarCuotaUnoCreditoRefinanciado(credito, comprobante, saldo, ticket, DiaCorte, fechaActual);
        //Aqui tengo que evaluar si no hubo excepciones para poder continuar
        if (!response.Completed)
            throw new CustomException("Error");
    }
}
```

En el método con las cuotas encontradas por el crédito que se está evaluando se busca cuáles de estas se encuentran o deberían encontrarse presupuestadas, si no se encuentra ninguna se evalúa si el crédito evaluado es de tipo refinanciado, ya que si lo es y no ha pasado por su primer corte el pago a esta primera cuota se procede a pagar completamente lo programado a pesar de no haber pasado por corte. Por lo que existe un método propio para pagar esta primera cuota de un crédito refinanciado.

Si existen cuotas presupuestadas se organizan en orden ascendente por fecha de corte, y de aquí en adelante se procede a usar la misma lógica que se hace en el momento de calcular el pago mínimo y total. Primera se evalúa si una de estas cuotas presupuestas tiene un abono en alguna cuota, si lo tuviera el saldo pendiente

de la cuota se iría pagando en el orden siguiente IVA mora, mora, IVA aval, aval, IVA interés, interés y por último el capital. Siempre este será el orden en que se aplique el pago. Si la cuota no tiene ningún abono se procede a realizar el pago de igual manera, pero ya utilizando los valores presupuestados.

```
(saldo, ivaMoraParcial) = AbonarPago(saldo, IvaMora);
(saldo, moraParcial) = AbonarPago(saldo, Mora);

(saldo, ivaAvalParcial) = AbonarPago(saldo, IvaAval);
(saldo, avalParcial) = AbonarPago(saldo, Aval);

(saldo, ivaInteresParcial) = AbonarPago(saldo, IvaInteres);
(saldo, interesParcial) = AbonarPago(saldo, Interes);

(saldo, capitalParcial) = AbonarPago(saldo, Capital);
```

Para realizar los pagos correspondientes se utiliza un método llamado AplicarPago que se llama en el orden indicado de cómo se debe aplicar el pago. Este será llamado en todo momento para saber en cada cuota y a cada ítem cuanto se debe abonar con respecto al saldo del pago que hay en el momento y cuanto debería pagar por cada ítem enviado.

```
public (decimal saldoRestante, decimal pagoEfectuado) AbonarPago(decimal saldo, decimal valorAPagar)
{
    var saldoNuevo = 0.0m;
    var pagoEfectuado = 0.0m;

    if (saldo > 0)
    {
        if (valorAPagar <= saldo)
        {
            pagoEfectuado = Math.Round(valorAPagar, 0, MidpointRounding.AwayFromZero);
            saldoNuevo = Math.Round(saldo - pagoEfectuado, 0, MidpointRounding.AwayFromZero);
        }
        else
        {
            pagoEfectuado = saldo;
            saldoNuevo = 0.0m;
        }
    }
    return (saldoNuevo, pagoEfectuado);
}
```

El método AbonarPago() recibirá dos parámetros que son el saldo actual en que se encuentra el pago, y el valor que se debería pagar con dicho saldo. Dentro se hace una validación, si el saldo es mayor a 0, si lo es se valida si el saldo alcanza a pagar lo del segundo parámetro, dependiendo de esto el *pagoEfectuado* es el completo que se quiere pagar o el saldo que queda. La información de cuanto saldo restante queda y cuanto fue lo que se pagó realmente de lo indicado será retornado por la función.

```
credito.monto_capital_pagado += capitalParcial;

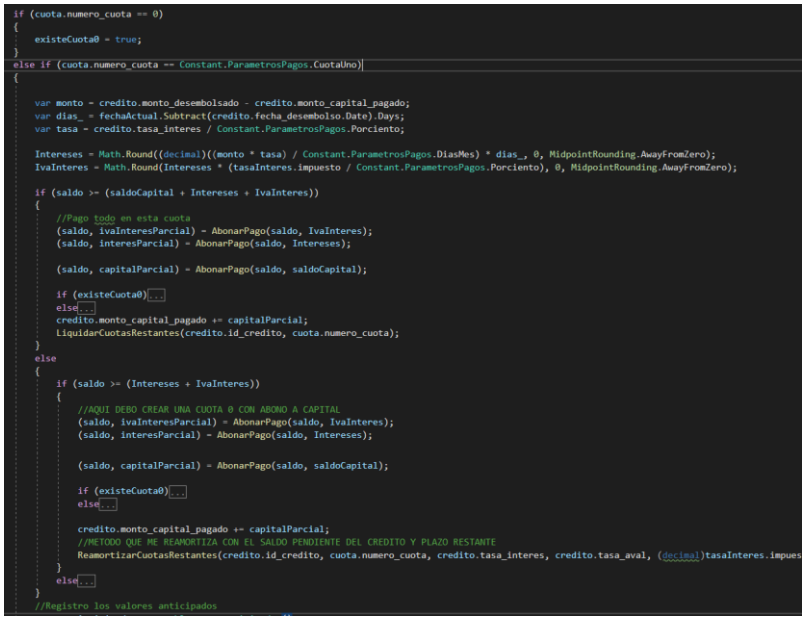
var diferencia = (cuota.valor_capital + cuota.valor_interes + cuota.valor_iva_interes + cuota.valor_aval + cuota.valor_iva_aval + cuota.valor_mora + cuota.valor_iva_mora)
    - (cuota.valor_capital_p + cuota.valor_interes_p + cuota.valor_iva_interes_p + cuota.valor_aval_p + cuota.valor_iva_aval_p + cuota.valor_mora_p + cuota.valor_iva_mora_p);

if (diferencia <= 0)
    cuota.pago_total = true;
else
    cuota.pago_total = false;


var detalleComprobante = new tbl_comprobante_detalles()
{
    fk_id_comprobante = comprobante.id_comprobante,
    fk_id_cuota = cuota.id_programacioncuotas,
    valor_credito = capitalParcial + interesParcial + ivaInteresParcial + avalParcial + ivaAvalParcial + moraParcial + ivaMoraParcial,
    valor_debito = 0.0m,
    valor_capital = capitalParcial,
    valor_interes = interesParcial,
    valor_iva_interes = ivaInteresParcial,
    valor_aval = avalParcial,
    valor_iva_aval = ivaAvalParcial,
    valor_mora = moraParcial,
    valor_iva_mora = ivaMoraParcial
};
Connection.ContextC10.tbl_comprobante_detalles.Add(detalleComprobante);
```

Los valores que devuelve el método de AbonarPago() son tratados como el pago real realizado a la cuota y es este valor el que se guarda en la tabla *tbl_programacion_cuotas*, después se valida si la diferencia que hay entre lo programado menos lo pagado hace que la cuota quede saldada, si es así, el estado de la cuota en pago_total queda con un valor en true, si no ha sido saldada por completo el estado de pago_total queda con un valor en false indicando que en dicha cuota se realizó un abono. El capital que se pagó se acumula en el campo de *monto_capital_pagado* del crédito y por último con la misma información del pago realizado se crea el detalle del comprobante donde se indica a que cuota y con que items fue asentado ese pago.

Para el segundo proceso que es de abono al capital de la deuda, se entraría si el pago mínimo del cliente fuera saldado por completo y adicional el saldo del pago fuera mayor a 1. Si se cumple estas condiciones se entraría al método de PagarCuotaAntesDeCorte(), la lógica para aplicar este pago se divide dependiendo del número de cuota en el que el crédito se encuentre. Si el número de cuota es el 0, solo se establece la variable existeCuota0 en true, y se continua para la próxima cuota que se encuentre antes del próximo corte del cliente, en este caso es la cuota 1.



```
if (cuota.numero_cuota == 0)
{
    existeCuota0 = true;
}
else if (cuota.numero_cuota == Constant.ParametrosPagos.CuotaUno)
{

    var monto = credito.monto_desembolsado - credito.monto_capital_pagado;
    var dias_ = fechaActual.Subtract(credito.fecha_desembolso.Date).Days;
    var tasa = credito.tasa_interes / Constant.ParametrosPagos.Porciento;

    Intereses = Math.Round((decimal)((monto * tasa) / Constant.ParametrosPagos.DiasMes) * dias_, 0, MidpointRounding.AwayFromZero);
    IvaInteres = Math.Round(Intereses * (tasaInteres.impuesto / Constant.ParametrosPagos.Porciento), 0, MidpointRounding.AwayFromZero);

    if (saldo >= (saldoCapital + Intereses + IvaInteres))
    {
        //Pago todo en esta cuota
        (saldo, ivaInteresParcial) = AbonarPago(saldo, IvaInteres);
        (saldo, interesParcial) = AbonarPago(saldo, Intereses);

        (saldo, capitalParcial) = AbonarPago(saldo, saldoCapital);

        if (existeCuota0)...
        else...
        credito.monto_capital_pagado += capitalParcial;
        LiquidarCuotasRestantes(credito.id_credito, cuota.numero_cuota);
    }
    else
    {
        if (saldo >= (Intereses + IvaInteres))
        {
            //AQUI DEBO CREAR UNA CUOTA 0 CON ABONO A CAPITAL
            (saldo, ivaInteresParcial) = AbonarPago(saldo, IvaInteres);
            (saldo, interesParcial) = AbonarPago(saldo, Intereses);

            (saldo, capitalParcial) = AbonarPago(saldo, saldoCapital);

            if (existeCuota0)...
            else...

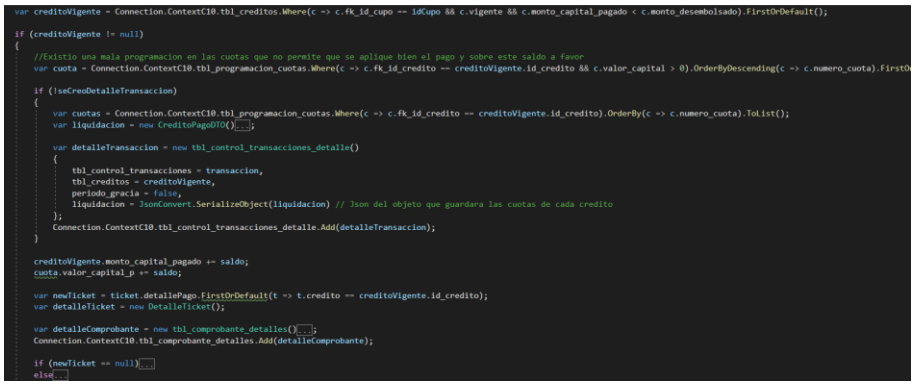
            credito.monto_capital_pagado += capitalParcial;
            //METODO QUE ME REAMORTIZA CON EL SALDO PENDIENTE DEL CREDITO Y PLAZO RESTANTE
            ReamortizarCuotasRestantes(credito.id_credito, cuota.numero_cuota, credito.tasa_interes, credito.tasa_aval, (decimal)tasaInteres.impues
        }
        else...
    }
    //Registro los valores anticipados
```

Si se encuentra en la cuota 1, se debe aplicar la misma lógica que en el cálculo del pago mínimo y total explicado anteriormente, en resumen, seria calcular los intereses anticipados del saldo del crédito desde la fecha desembolso hasta la fecha actual, calcular el IVA y asegurar primero el pago de estos dos items, después de pagar dichos items el saldo del pago restante deberá ser abonado al capital del crédito. En este punto ya se evalúa dos situaciones, la primera es que el saldo del pago permita pagar por completo los intereses anticipados calculados y el capital restante del crédito que al cumplir este caso se llamaría el método de *LiquidarCuotasRestantes()* que convertiría todos los valores siguientes en 0, o, si no alcanza a pagar todo el capital del crédito después de los intereses este llamara a el método *ReamortizarCuotasRestantes()*, que lo que hará es teniendo en cuenta el abono a capital realizado, las cuotas restantes con las que quedo el crédito se deberá reamortizar con respecto al saldo del crédito que quedo. En este caso, el pago se verá asentado en una cuota 0 que se deberá crear para no afectar el plazo del crédito y si ya existiera se acumularía el pago nuevo en sus valores.

En caso de que el numero de la cuota sea mayor a uno, igualmente se hará cobro de un interés anticipado el IVA y el saldo restante a capital, lo que difiera del proceso anterior es que el pago deberá quedar asentado en la cuota anterior en la que se encuentre el método, y se evaluara de la misma manera si después del abono se debería liquidar el crédito o se debería reamortizar.

```
//Condicion si me pago mas del valor total que adeuda, Solo deberia llegar si se paga por medio de consignacion
if (saldo >= 1)
{
    result = PagarSaldoAFavor(saldo, cupo.id_cupo, comprobante, transaccion, ticket, SeCreoDetalleTransaccion);
    if (!result.Completed)
        throw new CustomException("Error");
}
```

El tercer proceso general del método *AcentuarPagos()* se daría en el caso en que ya el cliente pago por completo su deuda y el saldo del pago sigue siendo mayor a 1, lo que implica que el cliente tendrá un saldo a favor. Este proceso ya no está dentro de un ciclo que recorra cada crédito porque se supone que en este punto ya todos los créditos se encuentran cancelados, pero, de igual manera dentro del método *PagarSaldoAFavor()* validara si esto se cumple o si hace falta un pago adicional que se podría dar por una mala amortización en el crédito.



```
var creditoVigente = Connection.ContextC10.tbl_creditos.Where(c => c.fk_id_cupo == idCupo && c.vigente && c.monto_capital_pagado < c.monto_desembolsado).FirstOrDefault();

if (creditoVigente != null)
{
    //Existio una mala programacion en las cuotas que no permite que se aplique bien el pago y sobre este saldo a favor
    var cuota = Connection.ContextC10.tbl_programacion_cuotas.Where(c => c.fk_id_credito == creditoVigente.id_credito && c.valor_capital > 0).OrderByDescending(c => c.numero_cuota).FirstO

    if (!seCreoDetalleTransaccion)
    {
        var cuotas = Connection.ContextC10.tbl_programacion_cuotas.Where(c => c.fk_id_credito == creditoVigente.id_credito).OrderBy(c => c.numero_cuota).ToList();
        var liquidacion = new CreditoPagoDTO()...;

        var detalleTransaccion = new tbl_control_transacciones_detalle()
        {
            tbl_control_transacciones = transaccion,
            tbl_creditos = creditoVigente,
            periodo_gracia = false,
            liquidacion = JsonConvert.SerializeObject(liquidacion) // Json del objeto que guardara las cuotas de cada credito
        };
        Connection.ContextC10.tbl_control_transacciones_detalle.Add(detalleTransaccion);
    }

    creditoVigente.monto_capital_pagado += saldo;
    cuota.valor_capital_p += saldo;

    var newTicket = ticket.detallePago.FirstOrDefault(t => t.credito == creditoVigente.id_credito);
    var detalleTicket = new DetalleTicket();

    var detalleComprobante = new tbl_comprobante_detalles()...;
    Connection.ContextC10.tbl_comprobante_detalles.Add(detalleComprobante);

    if (newTicket == null)...
    else...
```

En el método mencionado primero se valida si el cliente aún tiene un crédito vigente y que el capital pagado sea menor al desembolsado, si se encuentra un crédito con estas condiciones es que existiera un crédito mal amortizado, por lo que con el saldo restante se suma al capital del crédito como de la última cuota de este.

```
var credito = Connection.ContextC10.tbl_creditos.Where(c => c.fk_id_cupo == idCupo).OrderByDescending(c => c.fecha_desembolso).FirstOrDefault();
var cuota = Connection.ContextC10.tbl_programacion_cuotas.Where(c => c.fk_id_credito == credito.id_credito && c.valor_capital > 0).OrderByDescending(c => c.numero_cuota).FirstOrDefault

if (!seCreoDetalleTransaccion)
{
    var cuotas = Connection.ContextC10.tbl_programacion_cuotas.Where(c => c.fk_id_credito == credito.id_credito).OrderBy(c => c.numero_cuota).ToList();
    var liquidacion = new CreditoPagoDTO()
    {
        IdCredito = credito.id_credito,
        FechaDesembolso = credito.fecha_desembolso,
        MontoCapitalPagado = (decimal)credito.monto_capital_pagado,
        Vigencia = credito.vigente,
        EstadoCredito = credito.fk_id_estado,
        Cuotas = CustomParser.ParseList(cuotas)
    };

    var detalleTransaccion = new tbl_control_transacciones_detalle()
    {
        tbl_control_transacciones = transaccion,
        tbl_creditos = credito,
        periodo_gracia = false,
        liquidacion = JsonConvert.SerializeObject(liquidacion) // Json del objeto que guardara las cuotas de cada credito
    };
    Connection.ContextC10.tbl_control_transacciones_detalle.Add(detalleTransaccion);
}

credito.monto_capital_pagado += saldo;
cuota.valor_capital_p += saldo;

comprobante.observaciones = "Se aplico saldo a favor";
var newTicket = ticket.detallePago.FirstOrDefault(t => t.credito == credito.id_credito);
var detalleTicket = new DetalleTicket();

var detalleComprobante = new tbl_comprobante_detalles(){...};
Connection.ContextC10.tbl_comprobante_detalles.Add(detalleComprobante);
```

Si, por el contrario, no existiera ningún crédito vigente en este punto que sería el camino ideal, el saldo del pago que haya deberá quedar como un saldo a favor para el cliente, por lo que se busca el ultimo crédito desembolsado y se acumulara el valor restante del pago en el campo de monto_capital_pagado y a la última cuota del crédito para evidenciar allí ese saldo a favor.

Hasta aquí finalizaría todo el proceso que conlleva aplicar un pago en Credi 10 en tiendas, si todo es correcto se hará el commit de la transacción y finalizaría creando el ticket y notificando tanto al usuario con un mensaje en pantalla como al cliente mediante SMS y correo.

- **Consignación**



Esta vista tiene como ruta https://www.credi10.com.co/Pagos/Consignacion, el controlador de este es *PagosController* y para poder mostrar esta vista solo se hace la validación de si tiene o no permisos para poder entrar a ella. Con respecto a Pagos en tienda difiere en que aquí no se va a cargar una lista seleccionable ni tendrá un valor por defecto, en esta vista será obligatorio poner el código interno (CO) donde se quiera registrar la consignación. Con respecto al cliente si se hará la validación de que el ingresado tenga créditos vigentes por cancelar, si no los tuviera no le permitirá pasar a la siguiente pantalla y se obligara al usuario a registrar el pago por el proceso de cargue.

Cuando se da en aceptar se llamará al endpoint *CreateConsignacion()* que traerá la información del cliente y oficina que se ingresó en un primer momento, el valor del pago mínimo y total no se mostrara en esta pantalla ya que no es necesario visualizar esta información, en cambio, para aplicar el pago además del valor de este se necesita conocer en qué fecha fue la consignación, porque banco fue realizado y el tipo de transacción con el que fue hecho, estos dos últimos, se presentarán en pantalla por medio de un campo de selección donde los valores serán obtenidos desde base de datos de una manera parametrizada con la tabla *tbl_parametros_pagos.*



Cuando ingrese la información correspondiente al pago que se quiere realizar de tipo consignación, se le da en el botón de AplicarPago y este llamara por medio de POST el método CreateConsignacion(), este método solo tendrá dos validaciones, la primera *ValidateSession()* para verificar que el usuario que esté intentando ingresar al método tenga la sesión iniciado y además los permisos de hacer el pago y la segunda validación solo será que el valor ingresado sea mayor de 0, en esta pantalla si ingresa un valor mayor al pago total podrá ser registrado y será aplicado como saldo a favor el valor sobrante.

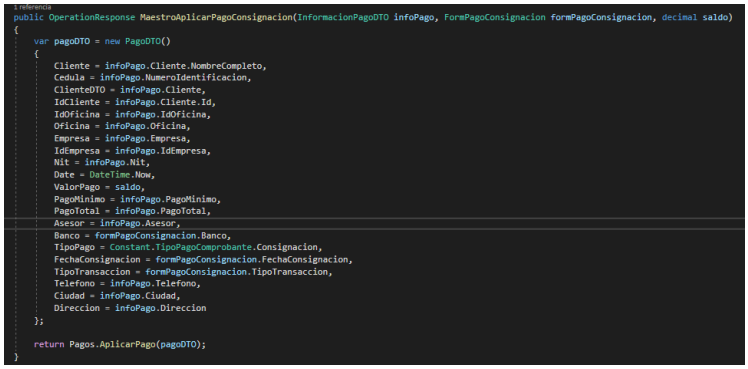
```
[HttpPost]
[ValidateAntiForgeryToken]
0 referencias
public ActionResult CreateConsignacion(InformacionPagoDTO infoPago, FormPagoConsignacion formPagoConsignacion)  //InformacionPagoDTO payment
{
    var result = new OperationResponse();

    try
    {
        var saldo = Convert.ToDecimal(formPagoConsignacion.ValorPago);

        if (!ValidateSession(true))
        {
            result.Status = false;
            result.Completed = false;
            result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.SinPermisos).Value;
        }
        else
        {
            if (saldo < 1)
            {
                throw new CustomException(ParameterManager.GetParamsms(Constant.MensajesKey.ValorNoPermitido).Value);
            }
            result = MPayment.MaestroAplicarPagoConsignacion(infoPago, formPagoConsignacion, saldo);
        }
    }
    catch (Exception ex)
    {
        result.Message = Constant.Mensajes.ErrorGenerico;
        result.Completed = false;
        result.IsError = true;
        result.Status = false;
    }
    Security.ControlAuditoria(SessionUser.UserName, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.CrearPago, result.Message, new { info = infoPago, consignacion = formPagoConsignacion});
    return Json(result);
}
```

Si las validaciones pasan, se procede a enviar la información recibida en el endpoint al método *MaestroAplicarPagoConsignacion()* que será el encargado de

organizar la información de la consignación en el objeto PagoDTO y comunicarse con el método de AplicarPago() el cual es el mismo método usado por pagos en tienda y el encargado de procesar por completo dicho pago. La diferencia entre pagos en tienda es que no es necesario enviar el contexto del controlador pues no se necesita crear ningún ticket ni enviar notificación al cliente sobre dicho proceso.



```
public OperationResponse MaestroAplicarPagoConsignacion(InformacionPagoDTO infoPago, FormPagoConsignacion formPagoConsignacion, decimal saldo)
{
    var pagoDTO = new PagoDTO()
    {
        Cliente = infoPago.Cliente.NombreCompleto,
        Cedula = infoPago.NumeroIdentificacion,
        ClienteDTO = infoPago.Cliente,
        IdCliente = infoPago.Cliente.Id,
        IdOficina = infoPago.IdOficina,
        Oficina = infoPago.Oficina,
        Empresa = infoPago.Empresa,
        IdEmpresa = infoPago.IdEmpresa,
        Nit = infoPago.Nit,
        Date = DateTime.Now,
        ValorPago = saldo,
        PagoMinimo = infoPago.PagoMinimo,
        PagoTotal = infoPago.PagoTotal,
        Asesor = infoPago.Asesor,
        Banco = formPagoConsignacion.Banco,
        TipoPago = Constant.TipoPagoComprobante.Consignacion,
        FechaConsignacion = formPagoConsignacion.FechaConsignacion,
        TipoTransaccion = formPagoConsignacion.TipoTransaccion,
        Telefono = infoPago.Telefono,
        Ciudad = infoPago.Ciudad,
        Direccion = infoPago.Direccion
    };

    return Pagos.AplicarPago(pagoDTO);
}
```

Hasta aquí finalizaría todo el proceso que conlleva aplicar un pago en Credi 10 por consignación, si todo es correcto se hará el commit de la transacción y finalizaría notificando al usuario con un mensaje en pantalla.

- **Cargue de Consignaciones**

Esta vista tiene como ruta https://www.credi10.com.co/Pagos/CarguePago, el controlador de este es *PagosController* y para poder mostrar esta vista solo se hace la validación de si tiene o no permisos para poder entrar a ella.



La vista tiene un formulario de 3 campos los cuales todos son requeridos, el primero y más importante es un archivo de Excel, este podrá ser subido desde los archivos del ordenador dando clic en el botón (Examinar el archivo) o se tendrá la opción de

arrastrar y soltar el archivo en la parte descrita. Este archivo solo podrá ser de formato .xlsx y pesar menos de 3 Mb aproximadamente. Los otros 2 campos serán la fecha que será un valor definido a la fecha actual y deshabilitado para que no se pueda cambiar y el punto de venta donde serán registrados los pagos subidos. El archivo Excel debe tener la siguiente estructura para su correcta lectura y posterior procesamiento.

	A	B	C	D	E
1	FECHA DD/MM/AAAA	CEDULA	VALOR	BANCO	MEDIO
2					

Al dar clic en el botón de Guardar después de haber llenado el formulario y subido el archivo se llamará por medio de POST el método *GuardarArchivo()* ubicado igualmente en PagosController.

```
[HttpPost]
[ValidateAntiForgeryToken]
0 referencias
public ActionResult GuardarArchivo(CargueArchivoDTO archivo, bool permiso = false)
{
    var result = new OperationResponse();

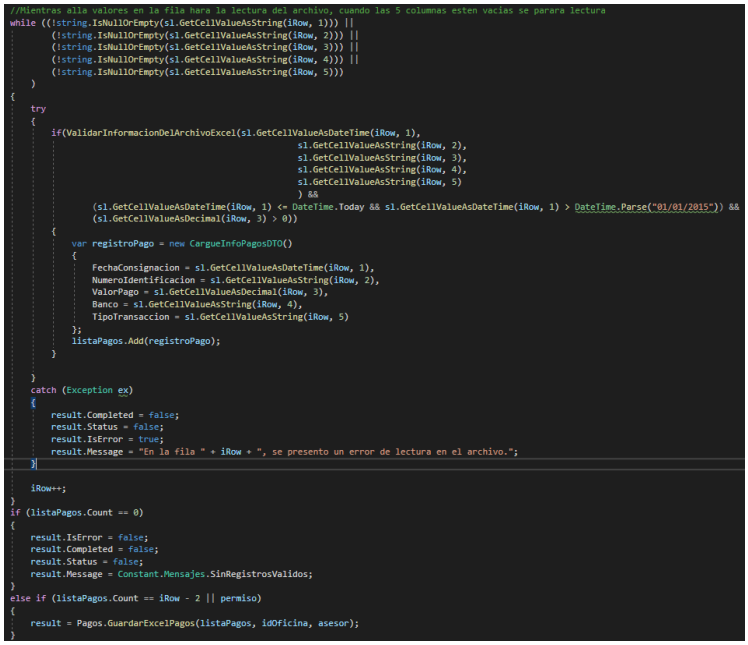
    if (!ModelState.IsValid)
    {
        TempData["estado"] = false;
        TempData["Message"] = Constant.General.ArchivoNoSubido;
        return RedirectToAction("CarguePago");
    }

    var codigo = archivo.CodigoInterno.Split('-');
    var oficina = Param.GetOficina(0, codigo[0]);

    //Si no igual a nulo hacer que redirija a incilio de sesion, esto puede pasar en el tiempo de espera.
    if (Session["user"] != null)
    {
        var json = Session["user"].ToString();
        var us = JsonConvert.DeserializeObject<UserDTO>(json);
        archivo.Asesor = us.UserName;
    }

    if (archivo.ArchivoPago.FileName.EndsWith(Constant.General.FormatoXls) || archivo.ArchivoPago.FileName.EndsWith(Constant.General.FormatoXlsx))
    {
        if (archivo.ArchivoPago.ContentLength < Constant.ParametrosPagos.Size)
        {
            archivo.IdOficina = oficina.IdOficina;
            result = MPayment.GuardarArchivoExcel(archivo, permiso);
        }
        else
        {
            result.Status = false;
            result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.ArchivoGrande).Value;
        }
    }
    else
    {
        result.Status = false;
        result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.ArchivoSinFormatoExcel).Value;
    }
    Security.ControlAuditoria(SessionUser.UserName, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.CargueArchivo, result.Message);
    return Json(result, JsonRequestBehavior.AllowGet);
}
```

El método validará que el objeto obtenido desde el formulario sea el valido, es que todos los valores requeridos tengan valor en ellos, si no lo fuera devolverá a redireccionar la vista con un mensaje informativo de que el archivo no se subió y en el primer llamado el valor de permiso no se enviará y tendrá su valor en falso por defecto. Después de esto se hará la validación de si el formato es de Excel y si este es menor al tamaño determinado de 3 Mb. Si las validaciones pasan se llamará el método *GuardarArchivoExcel()* encontrado en *MaestroPagosService* que será el encargado únicamente para guardar el archivo en el servidor para poder empezar su lectura, una vez guardado se llamará el método *LeerExcelPagos()*.



```
//Mientras alla valores en la fila hara la lectura del archivo, cuando las 5 columnas esten vacias se parara lectura
while ((!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 1))) ||
       (!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 2))) ||
       (!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 3))) ||
       (!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 4))) ||
       (!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 5)))
    )
{
    try
    {
        if(ValidarInformacionDelArchivoExcel(sl.GetCellValueAsDateTime(iRow, 1),
                                             sl.GetCellValueAsString(iRow, 2),
                                             sl.GetCellValueAsString(iRow, 3),
                                             sl.GetCellValueAsString(iRow, 4),
                                             sl.GetCellValueAsString(iRow, 5)
                                             ) &&
            (sl.GetCellValueAsDateTime(iRow, 1) <= DateTime.Today && sl.GetCellValueAsDateTime(iRow, 1) > DateTime.Parse("01/01/2015")) &&
            (sl.GetCellValueAsDecimal(iRow, 3) > 0))
        {
            var registroPago = new CargueInfoPagosDTO()
            {
                FechaConsignacion = sl.GetCellValueAsDateTime(iRow, 1),
                NumeroIdentificacion = sl.GetCellValueAsString(iRow, 2),
                ValorPago = sl.GetCellValueAsDecimal(iRow, 3),
                Banco = sl.GetCellValueAsString(iRow, 4),
                TipoTransaccion = sl.GetCellValueAsString(iRow, 5)
            };
            listaPagos.Add(registroPago);
        }

    }
    catch (Exception ex)
    {
        result.Completed = false;
        result.Status = false;
        result.IsError = true;
        result.Message = "En la fila " + iRow + ", se presento un error de lectura en el archivo.";
    }

    iRow++;
}
if (listaPagos.Count == 0)
{
    result.IsError = false;
    result.Completed = false;
    result.Status = false;
    result.Message = Constant.Mensajes.SinRegistrosValidos;
}
else if (listaPagos.Count == iRow - 2 || permiso)
{
    result = Pagos.GuardarExcelPagos(listaPagos, idOficina, asesor);
}
```

En la lectura del archivo se valida que mientras allá algún valor en la fila en una de las cinco primeras columnas se intentará leer los datos allí ingresados. Hasta que las 5 columnas se encuentren vacías el proceso de lectura se detendrá.

En el ciclo del while por cada fila se hará el llamado al método *ValidarInformacionDelArchivoExcel* enviando los valores de las primeras cinco columnas, el método explicado más adelante devolverá un valor verdadero si los valores contenidos en la fila son correctos para después agregar el registro en una lista de pagos, si el método devuelve falso en alguna fila este omitirá la fila, pero hará el conteo de esta. Cuando se acabe la lectura del archivo se validará primero que el contenido de esta lista sea mayor a cero y segundo que la cantidad de objetos en la lista sea igual a la cantidad de filas leídas en el archivo o que tenga el permiso en true recibido en el método.

En un primer momento este valor será falso, si se detecta que las filas leídas son diferentes de los registros guardados en la lista de pagos se le informara al usuario si quiere continuar con el proceso o si pienso cancelarlo para que suba otro archivo.



En caso de que el usuario de clic en Aceptar, este volverá a realizar el llamado al método que guardara y leerá el archivo, pero a diferencia de que enviara el valor de permiso en true para pasar la última validación y así omitir los valores inválidos y subir los demás valores para su posterior procesamiento.

```
try
{
    var carguePagos = new tbl_cargues_pagos(){...};
    Connection.ContextC10.tbl_cargues_pagos.Add(carguePagos);

    foreach (var consignacion in pagos)
    {
        //Validar si no existe un registro igual al que se hara
        var existeRegistro = Connection.ContextC10.sp_existe_registro_pago(consignacion.FechaConsignacion,
                consignacion.NumeroIdentificacion,
                consignacion.ValorPago,
                consignacion.Banco,
                consignacion.TipoTransaccion
            ).First();

        if (!existeRegistro.GetValueOrDefault())
        {
            var detalleCarguePagos = new tbl_cargues_pagos_detalle()
            {
                fk_id_cargue = carguePagos.id_carguepagos,
                fecha_consignacion = consignacion.FechaConsignacion,
                numero_identificacion = consignacion.NumeroIdentificacion,
                valor = consignacion.ValorPago,
                banco = consignacion.Banco,
                tipo_transaccion = consignacion.TipoTransaccion,
                error = Constant.Mensajes.NoIniciaProcesoPago,
                numero_intentos = 0

            };
            Connection.ContextC10.tbl_cargues_pagos_detalle.Add(detalleCarguePagos);
        }
        else
        {
            carguePagos.total_registros -= 1;
        }
    }

    if (carguePagos.total_registros == 0)
    {
        dbContextTransaction.Rollback();
        result.IsError = true;
        result.Status = false;
        result.Completed = false;
        result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.SinDatosParaRegistrar).Value;
        return result;
    }
    //Guardamos en la base de datos
    Connection.ContextC10.SaveChanges();
    //Hacemos commit de todos los datos si no hubo ningun error
    dbContextTransaction.Commit();

    result.Completed = true;
    result.Status = true;

    _ = Notification.EnviarCorreoSeguimientoPagos(asesor, carguePagos.id_carguepagos, carguePagos.total_registros, carguePagos.fecha_cargue.ToString("d"));
    idCargue = carguePagos.id_carguepagos;
    result.Message = String.Format(ParameterManager.GetParamsms(Constant.MensajesKey.ArchivoPagosCargado).Value, carguePagos.id_carguepagos.ToString());
}
```

Después de leer por completo el archivo de Excel y pasar el ultimo filtro de validaciones se obtiene la lista definitiva de los pagos que se deberían aplicar de tipo consignación, para

poder guardar la trazabilidad de este cargue y su posterior revisión se guarda el registro de este en la tabla *tbl_cargue_pagos* y luego se recorrerá mediante un ciclo la lista obtenida anteriormente, con los datos de allí se hará una última validación de que un pago no se haya subido y procesado anteriormente mediante el *sp_existe_registro_pago*, si no existe se guardara el detalle del pago en la tabla *tbl_cargue_pagos_detalle.* Si no se presenta ningún error se guardará la información en la base de datos haciendo el commit de la transacción y se enviará al correo del usuario que subió el archivo la información con el cargue para su posterior seguimiento.

```
if (result.Completed)
{
    Task.Run(async () =>
    {
        _ = await AplicarPagosAsync(idCargue, idOficina, asesor);
    });
}
```

Una vez finalice el guardado de los pagos que se deben aplicar se lanza una tarea asíncrona llamada *AplicarPagosAsync()* con el id del cargue que se realizó para empezar el procesamiento de los pagos y que el usuario no tenga que esperar en pantalla dicho procesamiento y pueda continuar con otros procesos.

```
var pagos = Connection.ContextC10.tbl_cargues_pagos_detalle.Where(c => c.fk_id_cargue == idCargue).ToList();

foreach (var pago in pagos)
{
    try
    {
        var cliente = Connection.ContextC10.tbl_clientes.FirstOrDefault(c => c.numero_identificacion == pago.numero_identificacion);
        var oficina = Connection.ContextC10.tbl_oficinas.FirstOrDefault(o => o.id_oficina == idOficina);

        if (cliente != null)
        {
            decimal minimo, total;
            (_, minimo, total, _, _) = Pagos.ConsultarPagoMinimoYTotal(cliente.id_clientes);
            //Me hace falta completar la informacion para crear el ticket correctamente
            var pagoDTO = new PagoDTO()
            {
                Cliente = cliente.nombre + " " + cliente.primer_apellido,
                Cedula = cliente.numero_identificacion,
                Telefono = oficina.telefono,
                Direccion = oficina.direccion,
                Ciudad = oficina.tbl_ciudades.descripcion,
                Oficina = oficina.tbl_marcas.nombre + " " + oficina.descripcion,
                Empresa = oficina.tbl_marcas.tbl_empresas.nombre,
                IdEmpresa = oficina.tbl_marcas.tbl_empresas.id_empresa,
                Nit = oficina.tbl_marcas.tbl_empresas.nit,
                IdCliente = cliente.id_clientes,
                IdOficina = idOficina,
                Date = DateTime.Now,
                ValorPago = pago.valor,
                Asesor = asesor,
                Banco = pago.banco,
                TipoPago = Constant.TipoPagoComprobante.Consignacion,
                FechaConsignacion = pago.fecha_consignacion,
                TipoTransaccion = pago.tipo_transaccion,
                PagoMinimo = minimo,
                PagoTotal = total
            };

            result = await AplicarPagoAsync(pagoDTO);
            if (result.Completed)
            {
                pago.error = null;
                pago.fk_id_comprobante = long.Parse(result.Data);
            }
            else
            {
                pago.error = Constant.Mensajes.ErrorProcesoPago;
            }
        }
        else
        {
            pago.error = Constant.Mensajes.NoExisteCliente;
        }
        Connection.ContextC10.SaveChanges();
    }
```

El método de AplicarPagosAsync() es un método asíncrono que procesará cada uno de los registros subidos en el cargue de Excel, este utilizará una copia fiel del método AplicarPago explicado anteriormente solo que con la diferencia de que será asíncrono. Al aplicar el pago por cada registro en el detalle del cargue, se guardará el comprobante asociado si fuera exitoso el pago y si no guardara el error del porque no se pudo aplicar el pago.

- **Cargue de Pagos PayU**

Esta vista tiene como ruta https://www.credi10.com.co/Pagos/CarguePagosPayU, el controlador de este es *PagosController* y para poder mostrar esta vista solo se hace la validación de si tiene o no permisos para poder entrar a ella.



La vista tiene un formulario de solo un campo requerido, un archivo de Excel que contendrá los registros de pagos por PayU que por alguna razón no han sido aplicados a credi10 de manera automática. Este podrá subirse desde los archivos del ordenador dando clic en el botón (Examinar el archivo) o se podrá arrastrar y soltar el archivo en la parte descrita. Este archivo solo podrá ser de formato .xlsx y pesar menos de 3 Mb aproximadamente. El archivo Excel debe tener la siguiente estructura para su correcta lectura y posterior procesamiento.

A	B	C	D
Cedula	ValorPago	Fecha Pago DD/MM/AAAA	Medio de Pago

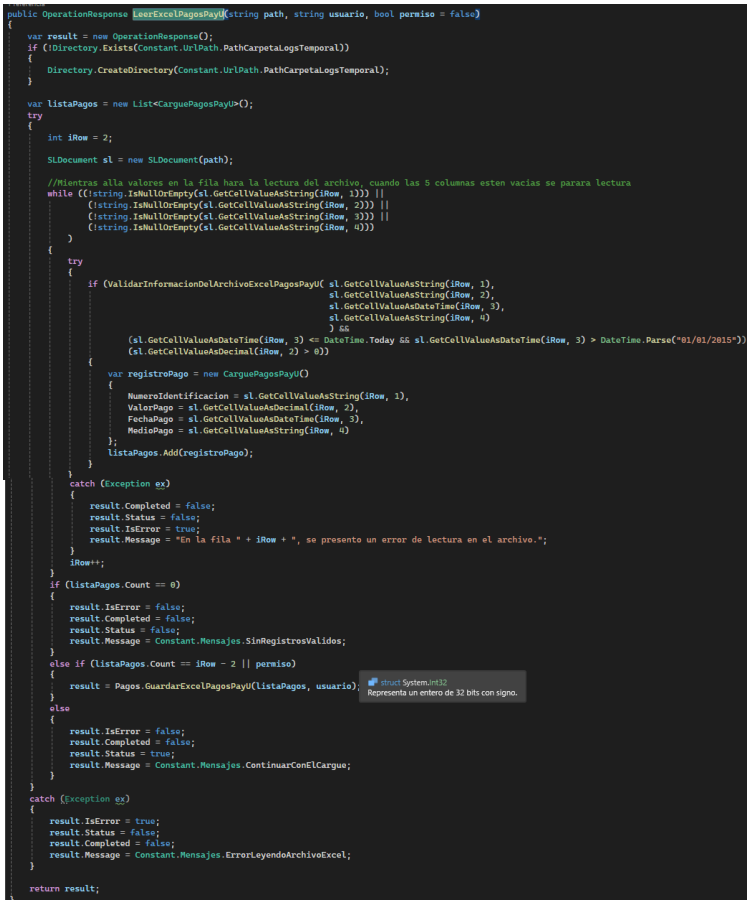
Al dar clic en el botón de Guardar después de haber subido el archivo se llamará por medio de POST el método *GuardarArchivoPagosPayU()* ubicado igualmente en PagosController.

```
[HttpPost]
[ValidateAntiForgeryToken]
0 referencias
public ActionResult GuardarArchivoPagosPayU(CargueArchivoPagosPayU Archivo, bool permiso = false)
{
    var result = new OperationResponse();
    string usuario;
    if (!ModelState.IsValid)
    {
        result.Completed = false;
        result.Status = false;
        result.Message = Constant.General.ArchivoNoSubido;
        return Json(result);
    }

    //Si es igual a nulo hacer que redirija a inicio de sesion, esto puede pasar en el tiempo de espera.
    if (Session["user"] != null)
    {
        var json = Session["user"].ToString();
        var us = JsonConvert.DeserializeObject<UserDTO>(json);
        usuario = us.UserName;
    }
    else
    {
        result.Completed = false;
        result.Status = false;
        result.Message = "Usuario desconectado";
        return Json(result);
    }

    if (Archivo.ArchivoPago.FileName.EndsWith(Constant.General.FormatoXls) || Archivo.ArchivoPago.FileName.EndsWith(Constant.General.FormatoXlsx
    {
        if (Archivo.ArchivoPago.ContentLength < Constant.ParametrosPagos.Size)
        {
            result = MPayment.GuardarArchivoExcelPagosPayU(Archivo, usuario, permiso);
        }
        else
        {
            result.Status = false;
            result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.ArchivoGrande).Value;
        }
    }
    else
    {
        result.Status = false;
        result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.ArchivoSinFormatoExcel).Value;
    }
    Security.ControlAuditoria(SessionUser.UserName, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.CargueArchivoPagosPayU, result.Messag
    return Json(result, JsonRequestBehavior.AllowGet);
```

El método recibe el valor de permiso en falso por defecto en su primer llamado junto al archivo subido. Se hará la validación de si el formato es de Excel y si este es menor al tamaño determinado de 3 Mb. Si las validaciones pasan se llamará el método *GuardarArchivoExcelPagosPayU()* encontrado en *MaestroPagosService* que será el encargado únicamente para guardar el archivo en el servidor para poder empezar su lectura, una vez guardado se llamará el método *LeerExcelPagosPayU().*



```
public OperationResponse LeerExcelPagosPayU(string path, string usuario, bool permiso = false)
{
    var result = new OperationResponse();
    if (!Directory.Exists(Constant.UrlPath.PathCarpetaLogsTemporal))
    {
        Directory.CreateDirectory(Constant.UrlPath.PathCarpetaLogsTemporal);
    }

    var listaPagos = new List<CarguePagosPayU>();
    try
    {
        int iRow = 2;

        SLDocument sl = new SLDocument(path);

        //Mientras alla valores en la fila hara la lectura del archivo, cuando las 5 columnas esten vacias se parara lectura
        while ((!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 1))) ||
               (!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 2))) ||
               (!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 3))) ||
               (!string.IsNullOrEmpty(sl.GetCellValueAsString(iRow, 4)))
            )
        {
            try
            {
                if (ValidarInformacionDelArchivoExcelPagosPayU( sl.GetCellValueAsString(iRow, 1),
                                                                sl.GetCellValueAsString(iRow, 2),
                                                                sl.GetCellValueAsDateTime(iRow, 3),
                                                                sl.GetCellValueAsString(iRow, 4)
                                                                ) &&
                    (sl.GetCellValueAsDateTime(iRow, 3) <= DateTime.Today && sl.GetCellValueAsDateTime(iRow, 3) > DateTime.Parse("01/01/2015")) &&
                    (sl.GetCellValueAsDecimal(iRow, 2) > 0))
                {
                    var registroPago = new CarguePagosPayU()
                    {
                        NumeroIdentificacion = sl.GetCellValueAsString(iRow, 1),
                        ValorPago = sl.GetCellValueAsDecimal(iRow, 2),
                        FechaPago = sl.GetCellValueAsDateTime(iRow, 3),
                        MedioPago = sl.GetCellValueAsString(iRow, 4)
                    };
                    listaPagos.Add(registroPago);
                }
            }
            catch (Exception ex)
            {
                result.Completed = false;
                result.Status = false;
                result.IsError = true;
                result.Message = "En la fila " + iRow + ", se presento un error de lectura en el archivo.";
            }
            iRow++;
        }
        if (listaPagos.Count == 0)
        {
            result.IsError = false;
            result.Completed = false;
            result.Status = false;
            result.Message = Constant.Mensajes.SinRegistrosValidos;
        }
        else if (listaPagos.Count == iRow - 2 || permiso)
        {
            result = Pagos.GuardarExcelPagosPayU(listaPagos, usuario);
        }
        else
        {
            result.IsError = false;
            result.Completed = false;
            result.Status = true;
            result.Message = Constant.Mensajes.ContinuarConElCargue;
        }
    }
    catch (Exception ex)
    {
        result.IsError = true;
        result.Status = false;
        result.Completed = false;
        result.Message = Constant.Mensajes.ErrorLeyendoArchivoExcel;
    }

    return result;
}
```

struct System.Int32
Representa un entero de 32 bits con signo.

En la lectura del archivo se valida que mientras allá algún valor en la fila en una de las cuatro primeras columnas se intentará leer los datos allí ingresados. Hasta que las cuatro columnas se encuentren vacías el proceso de lectura se detendrá.

En el ciclo del while por cada fila se hará el llamado al método *ValidarInformacionDelArchivoExcelPagosPayU* enviando los valores de las primeras cinco columnas, el método explicado más adelante devolverá un valor verdadero si los valores contenidos en la fila son correctos para después agregar el registro en una lista de pagos, si el método devuelve falso en alguna fila este omitirá la fila, pero hará el conteo de esta. Cuando se acabe la lectura del archivo se validará primero que el contenido de esta lista sea mayor a cero y segundo que la cantidad de objetos en la lista sea igual a la cantidad de filas leídas en el archivo o que tenga el permiso en true recibido en el método.

En un primer momento este valor será falso, si se detecta que las filas leídas son diferentes de los registros guardados en la lista de pagos se le informara al usuario si quiere continuar con el proceso o si pienso cancelarlo para que suba otro archivo.



En caso de que el usuario de clic en Aceptar, este volverá a realizar el llamado al método que guardara y leerá el archivo, pero a diferencia de que enviara el valor de permiso en true para pasar la última validación y así omitir los valores inválidos y subir los demás valores para su posterior procesamiento.

```
try
{
    var carguePagos = new tbl_cargues_pagos()
    {
        total_registros = pagos.Count,
        fk_id_oficina = 298,
        fecha_cargue = DateTime.Now,
        usuario_creacion = usuario
    };
    Connection.ContextC10.tbl_cargues_pagos.Add(carguePagos);

    foreach (var pago in pagos)
    {
        //Validar si no existe un registro igual al que se hara
        var existeRegistro = Connection.ContextC10.sp_existe_registro_pago(pago.FechaPago,
                pago.NumeroIdentificacion,
                pago.ValorPago,
                "PayU",
                pago.MedioPago
            ).First();

        if (!existeRegistro.GetValueOrDefault())
        {
            var detalleCarguePagos = new tbl_cargues_pagos_detalle()
            {
                fk_id_cargue = carguePagos.id_carguepagos,
                fecha_consignacion = pago.FechaPago,
                numero_identificacion = pago.NumeroIdentificacion,
                valor = pago.ValorPago,
                banco = "PayU",
                tipo_transaccion = pago.MedioPago,
                error = Constant.Mensajes.NoIniciaProcesoPago,
                numero_intentos = 0
            };
            Connection.ContextC10.tbl_cargues_pagos_detalle.Add(detalleCarguePagos);
        }
        else
        {
            carguePagos.total_registros -= 1;
        }
    }

    if (carguePagos.total_registros == 0)
    {
        dbContextTransaction.Rollback();
        result.IsError = true;
        result.Status = false;
        result.Completed = false;
        result.Message = ParameterManager.GetParamsms(Constant.MensajesKey.SinDatosParaRegistrar).Value;
        return result;
    }
    //Guardamos en la base de datos
    Connection.ContextC10.SaveChanges();
    //Hacemos commit de todos los datos si no hubo ningun error
    dbContextTransaction.Commit();

    result.Completed = true;
    result.Status = true;

    idCargue = carguePagos.id_carguepagos;
    result.Message = String.Format(ParameterManager.GetParamsms(Constant.MensajesKey.ArchivoPagosCargado).Value, carguePagos.id_carguepagos.ToString());
}
```

Después de leer por completo el archivo de Excel y pasar el ultimo filtro de validaciones se obtiene la lista definitiva de los pagos que se deberían aplicar de tipo consignación, para poder guardar la trazabilidad de este cargue y su posterior revisión se guarda el registro de este en la tabla *tbl_cargue_pagos* y luego se recorrerá mediante un ciclo la lista obtenida anteriormente, con los datos de allí se hará una última validación de que un pago no se haya subido y procesado anteriormente mediante el *sp_existe_registro_pago*, si no existe se guardara el detalle del pago en la tabla *tbl_cargue_pagos_detalle.* Si no se presenta ningún error se guardará la información en la base de datos haciendo el commit de la transacción.

```
if (result.Completed)
{
    Task.Run(async () =>
    {
        _ = await AplicarPagosPayUAsync(idCargue, usuario);
    });
}
```

Una vez finalice el guardado de los pagos que se deben aplicar se lanza una tarea asíncrona llamada *AplicarPagosPayUAsync()* con el id del cargue que se realizó para empezar el procesamiento de los pagos y que el usuario no tenga que esperar en pantalla dicho procesamiento.



```
1 reference
public async Task<OperationResponse> AplicarPagosPayUAsync(int idCargue, string asesor)
{
    var result = new OperationResponse();
    var pagos = Connection.ContextC10.tbl_cargues_pagos_detalle.Where(c => c.fk_id_cargue == idCargue).ToList();

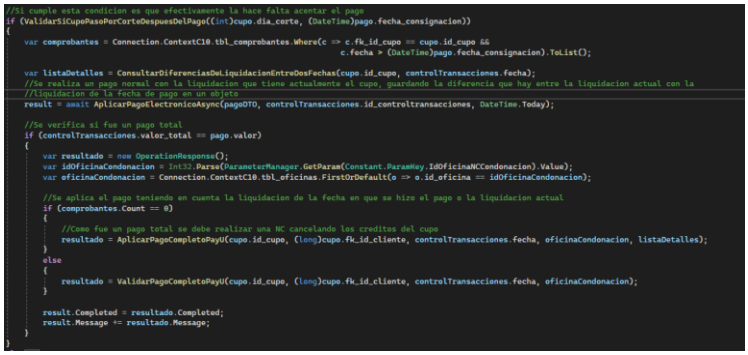
    foreach (var pago in pagos)
    {
        try
        {
            var cliente = Connection.ContextC10.tbl_clientes.FirstOrDefault(c => c.numero_identificacion == pago.numero_identificacion);
            var idOficina = Int32.Parse(ParameterManager.GetParam(Constant.ParamKey.OficinaPagosWeb).Value);
            var oficina = Connection.ContextC10.tbl_oficinas.FirstOrDefault(o => o.id_oficina == idOficina);
            if (cliente != null)
            {
                var pagoDTO = new PagoDTO()
                {
                    Cliente = cliente.nombre + " " + cliente.primer_apellido,
                    Cedula = cliente.numero_identificacion,
                    Telefono = oficina.telefono,
                    Direccion = oficina.direccion,
                    Ciudad = oficina.tbl_ciudades.descripcion,
                    Oficina = oficina.tbl_marcas.nombre + " " + oficina.descripcion,
                    Empresa = oficina.tbl_marcas.tbl_empresas.nombre,
                    IdEmpresa = oficina.tbl_marcas.tbl_empresas.id_empresa,
                    Nit = oficina.tbl_marcas.tbl_empresas.nit,
                    IdCliente = cliente.id_clientes,
                    IdOficina = oficina.id_oficina,
                    Date = DateTime.Now,
                    ValorPago = pago.valor,
                    Asesor = asesor,
                    Banco = pago.banco,
                    TipePago = Constant.TipoPagoComprobante.Online,
                    FechaConsignacion = pago.fecha_consignacion,
                    TipeTransaccion = pago.tipo_transaccion
                };

                var cupo = Connection.ContextC10.tbl_cupos.FirstOrDefault(c => c.fk_id_cliente == cliente.id_clientes);
                var controlTransacciones = Connection.ContextC10.tbl_control_transacciones.Where(c => c.fecha == (DateTime)pago.fecha_consignacion &&
                                                                                                  c.fk_id_cupo == cupo.id_cupo).OrderByDescending(c => c.id_controltransacciones).FirstOrDefault();

                if(controlTransacciones != null && controlTransacciones.fk_id_comprobante == null)
                {
                    if(controlTransacciones.fk_id_comprobante == null)
                    {
                        //Si cumple esta condicion es que efectivamente la hace falta acentar el pago
                        if (ValidarSiCupoPasoPorCorteDespuesDelPago((int)cupo.dia_corte, (DateTime)pago.fecha_consignacion))...
                        else...
                        if (result.Completed)
                        {
                            pago.error = null;
                            pago.fk_id_comprobante = long.Parse(result.Data);
                        }
                        else
                        {
                            pago.error = result.Message;
                        }
                    }
                    else
                    {
                        pago.error = "El pago ya se encuentra asentado.";
                    }
                }
                else
                {
                    pago.error = "No se encontro una transaccion para la fecha indicada.";
                }
            }
            else
            {
                pago.error = Constant.Mensajes.NoExisteCliente;
            }
            Connection.ContextC10.SaveChanges();
        }
        catch (Exception e)
        {
            result.IsError = true;
            result.Completed = false;
            result.Message = "Error en el proceso de aplicar el pago. Exception:" + e.InnerException;
        }
    }
    return result;
}
```

El método de AplicarPagosAsync() es un método asíncrono que procesará cada uno de los registros subidos en el cargue de Excel, este proceso lo que hace es buscar si existe un registro en la tabla de control_transacciones del cliente y la fecha que se subió en el archivo, si este se encuentra se validó que el pago no tenga un comprobante asociado, el cual significaría que el pago ya fue asentado y no se aplicara este nuevamente. Si cumple estas condiciones es que el pago falta por asentarse y se hará otras series de validaciones para saber cómo aplicar el pago.

Primero se tendrá que identificar si la transacción realizada por payU fue antes del último corte del cliente, si esto se cumple, se aplica el pago llamando al método *AplicarPagoElectronicoAsync()* de manera síncrona, con el valor del pago subido en el archivo. Con la transacción se valida si el cliente aplico el pago total o un abono, si fue el pago total y este paso por corte se debe crear una nota de crédito para ajustar los valores y así quede cancelado el crédito. Para garantizar que el pago total realizado si cancele su deuda se valida si el cliente no realizo pagos posteriores, y si los hizo, que se garantice la cancelación de la deuda.



```
//Si cumple esta condicion es que efectivamente la hace falta acentar el pago
if (ValidarSiCupoPasoPorCorteDespuesDelPago((int)cupo.dia_corte, (DateTime)pago.fecha_consignacion))
{
    var comprobantes = Connection.ContextC10.tbl_comprobantes.Where(c => c.fk_id_cupo == cupo.id_cupo &&
                                                                         c.fecha > (DateTime)pago.fecha_consignacion).ToList();

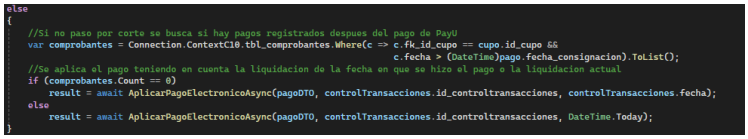
    var listaDetalles = ConsultarDiferenciasDeLiquidacionEntreDosFechas(cupo.id_cupo, controlTransacciones.fecha);
    //Se realiza un pago normal con la liquidacion que tiene actualmente el cupo, guardando la diferencia que hay entre la liquidacion actual con la
    //liquidacion de la fecha de pago en un objeto
    result = await AplicarPagoElectronicoAsync(pagoDTO, controlTransacciones.id_controltransacciones, DateTime.Today);

    //Se verifica si fue un pago total
    if (controlTransacciones.valor_total == pago.valor)
    {
        var resultado = new OperationResponse();
        var idOficinaCondonacion = Int32.Parse(ParameterManager.GetParam(Constant.ParamKey.IdOficinaNCCondonacion).Value);
        var oficinaCondonacion = Connection.ContextC10.tbl_oficinas.FirstOrDefault(o => o.id_oficina == idOficinaCondonacion);

        //Se aplica el pago teniendo en cuenta la liquidacion de la fecha en que se hizo el pago o la liquidacion actual
        if (comprobantes.Count == 0)
        {
            //Como fue un pago total se debe realizar una NC cancelando los creditos del cupo
            resultado = AplicarPagoCompletoPayU(cupo.id_cupo, (long)cupo.fk_id_cliente, controlTransacciones.fecha, oficinaCondonacion, listaDetalles);
        }
        else
        {
            resultado = ValidarPagoCompletoPayU(cupo.id_cupo, (long)cupo.fk_id_cliente, controlTransacciones.fecha, oficinaCondonacion);
        }

        result.Completed = resultado.Completed;
        result.Message += resultado.Message;
    }
}
```

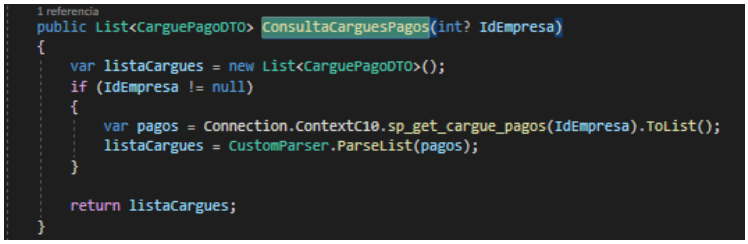
Si desde la fecha de la transacción hasta la fecha actual el cliente no ha pasado por corte este pago se aplicará con el valor subido en el archivo, y teniendo en cuenta si el cliente tenga o no pagos posteriores este se asentará con respecto a la fecha de la transacción o la fecha actual, esto hará la diferencia en algunos valores a cobrar. Después de aplicar el pago no se validará algo adicional y allí el proceso terminará.



```
else
{
    //Si no paso por corte se busca si hay pagos registrados despues del pago de PayU
    var comprobantes = Connection.ContextC10.tbl_comprobantes.Where(c => c.fk_id_cupo == cupo.id_cupo &&
                                                                         c.fecha > (DateTime)pago.fecha_consignacion).ToList();
    //Se aplica el pago teniendo en cuenta la liquidacion de la fecha en que se hizo el pago o la liquidacion actual
    if (comprobantes.Count == 0)
        result = await AplicarPagoElectronicoAsync(pagoDTO, controlTransacciones.id_controltransacciones, controlTransacciones.fecha);
    else
        result = await AplicarPagoElectronicoAsync(pagoDTO, controlTransacciones.id_controltransacciones, DateTime.Today);
}
```

- **Seguimiento Cargue Consignaciones**

Para realizar el seguimiento y tener una trazabilidad de todos los cargues realizados en credi10 esta creada una vista con la ruta https://www.credi10.com.co/Pagos/EstadoCargue, para ingresar a ella también se debe contar con los permisos y accesos necesarios por el usuario ya que dentro del controlador también se valida la sesión, el endpoint llamara el método *ConsultaCarguesPagos()* y enviara como parámetro el id de la empresa con el que está asociado el usuario de la consulta, este método consulta un procedimiento almacenado con la lista de cargues realizados de la misma empresa del usuario.

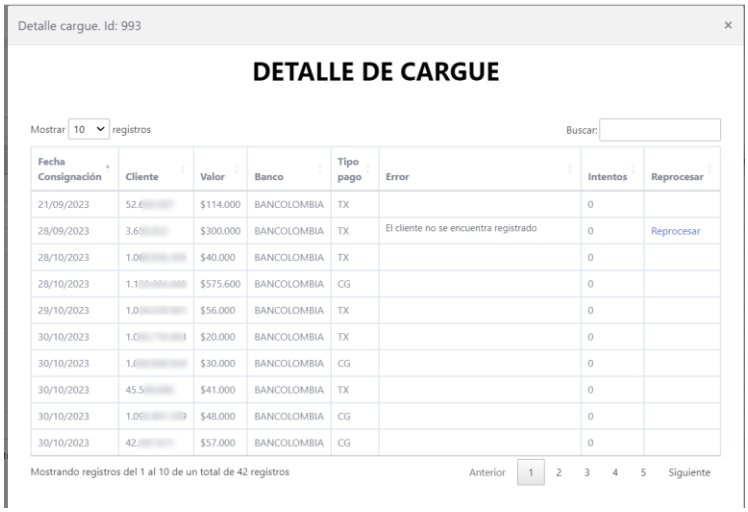


```
1 referencia
public List<CarguePagoDTO> ConsultaCarguesPagos(int? IdEmpresa)
{
    var listaCargues = new List<CarguePagoDTO>();
    if (IdEmpresa != null)
    {
        var pagos = Connection.ContextC10.sp_get_cargue_pagos(IdEmpresa).ToList();
        listaCargues = CustomParser.ParseList(pagos);
    }

    return listaCargues;
}
```

En la vista se listarán los cargues devueltos por la consulta, organizados descendentemente con respecto a la fecha del cargue, en ella se mostrará la información del usuario que hizo el cargue, la fecha, el total de registros y cuantos de estos se aplicó exitosamente o hubo errores.





En la opción detalle se podrá evidenciar ya todos los registros a detalle del cargue seleccionado, se mostrara la información que contenía el Excel subido, como la cedula del cliente, el valor del pago y demás datos necesarios de la consignación, habrá una columna de error, intentos y Reprocesar, allí se mostrara información adicional del procesamiento del pago por si hubo algún error, cuantos intentos se han realizado por cada uno (el valor por defecto estará en 0 y si no hay error en el primer intento se quedara dicho valor) y si existiera un error relacionado en la columna de Reprocesar aparecer un botón que cumple la función del verbo relacionado.

```
public OperationResponse ReprocesarPago(int idPago, string asesor)
{
    var result = new OperationResponse();

    try
    {
        var consignacion = Connection.ContextC10.tbl_cargues_pagos_detalle.FirstOrDefault(c => c.id_carguedetalle == idPago);
        var datosCargue = Connection.ContextC10.tbl_cargues_pagos.FirstOrDefault(c => c.id_carguepagos == consignacion.fk_id_cargue);
        var infoCliente = Connection.ContextC10.sp_get_datos_clientes(consignacion?.numero_identificacion).First();
        var oficina = Connection.ContextC10.tbl_oficinas.FirstOrDefault(o => o.id_oficina == (int)datosCargue.fk_id_oficina);

        decimal minimo, total;
        (_, minimo, total, _, _) = Pagos.ConsultarPagoMinimoYTotal((long)infoCliente.Id);

        var pagoDTO = new PagoDTO()
        {
            IdCliente = (long)infoCliente.Id,
            Cliente = infoCliente.Nombre,
            IdOficina = (int)datosCargue.fk_id_oficina,
            IdEmpresa = oficina.tbl_marcas.fk_id_empresa,
            Date = DateTime.Now,
            ValorPago = consignacion.valor,
            Asesor = asesor,
            Banco = consignacion.banco,
            TipoPago = Constant.TipoPagoComprobante.Consignacion,
            FechaConsignacion = consignacion.fecha_consignacion,
            TipoTransaccion = consignacion.tipo_transaccion,
            PagoTotal = total,
            PagoMinimo = minimo
        };

        result = AplicarPago(pagoDTO);

        if (result.Completed)
        {
            result.Status = true;
            consignacion.error = null;
            consignacion.numero_intentos++;
            consignacion.fk_id_comprobante = long.Parse(result.Data);
```

Cuando se dé clic en la opción de reprocesar, la acción llegara al método ubicado en PagosService llamada *ReprocesarPago()*, este recibirá como parámetro el id del detalle del cargue en donde se quiere reintentar el proceso y el usuario quien está haciendo el intento el cual será llamado asesor, el método será encargado de construir el mismo objeto necesario con los datos del pago para poder llamar al método síncrono AplicarPago() explicado con anterioridad. Si el pago se aplica con éxito el error del detalle se volverá null y se le asignará a la consignación el comprobante que se realizó para su trazabilidad.

El archivo de pagos PayU también se podrá visualizar en esta vista, la única diferencia que tendrá con respecto a las consignaciones es que los subidos por el cargue de pagos PAyU no tendrá la opción de reprocesar el pago, en tal caso de haber tenido error al aplicar el pago por una u otra razón se debe subir nuevamente un archivo con el registro que no se aplicó teniendo en cuenta que lo que causo el error ya ha sido solucionado.

1.3.4.1 Otros procesos

En Creditos.BLL existirán diferentes métodos que tienen una funcionalidad especifica que ayuda a cumplir con diferentes requerimientos menores para el módulo completo de pagos. Estos métodos son usados en diferentes momentos por el proyecto Creditos.Web.Admin.

A. **Buscar punto de venta y validar si este existe.**

```
function buscarOficina() {
    var oficina = $("#CodigoInterno").val();
    if (oficina != "") {
        $.ajax({
            url: "@Url.Action("BuscarPuntoVenta", "Oficinas")",
            data: { descripcion: oficina},
            type: "GET",
            success: function (respuesta) {
                if (String(respuesta.Status).toLowerCase() === "false") {
                    $('#CodigoInterno').val("")
                    MostrarMensaje("La oficina no se encuentra registrada")
                    $('#validar').prop("disabled", true);
                    document.querySelector('#validar').style.display = 'none';
                } else {
                    oficinaExiste = true;
                    $('#CodigoInterno').val(respuesta.Data)
                }
                habilitarbtnValidar()
            }
        }).fail(function (xhr, status, error) {
            MostrarMensaje("Error en la consulta. Revisa la conexión a internet")
        })
    }
}
```

Esta función está incluida en todas las vistas del módulo de pagos que tenga incluido el campo de ingresar el código interno de un punto de venta, será el encargado de llamar un endpoint que validara si el valor ingresado pertenece o no para un punto de venta. Si es así, pondrá el co y nombre de la oficina en el campo, si no existiera la oficina se mostrará un mensaje informando al usuario.

B. Buscar cliente y validar si tiene créditos pendientes por pagar.

```
function buscarDocumento() {
    var numeroDocumento = $("#NumeroIdentificacion").val();
    if (numeroDocumento != "") {
        var auxNumeroDocumento = numeroDocumento.replaceAll(",", "");
        auxNumeroDocumento = auxNumeroDocumento.replaceAll(".", "");
        $.ajax({
            url: "@Url.Action("PagosPendientes", "Pagos")",
            data: { numero: auxNumeroDocumento},
            type: "GET",
            success: function (respuesta) {

                if (respuesta.StatusCliente) {
                    $("#NombreCliente").text(respuesta.NombreCompleto);
                    clienteExiste = true;
                } else {
                    clienteExiste = false;
                    MostrarMensaje(respuesta.Mensaje)
                    auxInputNumeroDocumento.set("");
                    $("#NombreCliente").text("");
                }

                habilitarbtnValidar()
            }
        }).fail(function (xhr, status, error) {
            MostrarMensaje("Error en la consulta. Revisa la conexión a internet")
        })
    }
}
```

Esta función está incluida en todas las vistas del módulo de pagos que tenga incluido el campo de ingresar el número de identificación del cliente, será el encargado de llamar un endpoint que validara si el cliente existe y si este tiene créditos pendientes por pagar, si cumple con las dos validaciones, en la vista se cargara el nombre y permitirá continuar hacia la otra vista. Si no, se mostrará un mensaje informando al usuario de la validación realizada.

C. Validar la sesión del usuario

```
protected bool ValidateSession(bool IsPage)
{
    if (Session["user"] != null)
    {
        var json = Session["user"].ToString();
        var us = JsonConvert.DeserializeObject<UserDTO>(json);
        sessionUser = us;
        Permit = new PermitDTO();
        var url = Request.Url.AbsolutePath;
        if (url != null)
        {
            if (!string.IsNullOrEmpty(url) && url != "/")
            {
                Permit = Security.Validate(url, us.Perfiles);
            }
            if (!Permit.IsGrant && url != "/" && !url.Contains("Login") && !url.Contains("Home"))
            {
                return false;
            }
        }
        this.CurrentUser = us.Id;

        ViewBag.CambioClave = us.CambioClave;
        ViewBag.Permit = Permit;
        ViewBag.PermitSeguridad = Security.ValidatePaginaDependiente(Constant.PermisosBotones.BotonSeguridadPass, SessionUser.Perfiles);
        ViewBag.PermitInscripcionProspecto = Security.ValidatePaginaDependiente(Constant.PaginasPrincipales.InscripcionProspecto, SessionUser.Perfiles);
        ViewBag.User = us;
        ViewBag.Menu = Security.GetMenu(us.Perfiles);
        ViewBag.ColorPersonalizado = ParameterManager.GetParam(SkinTheme.ThemeColor.ToString()).Value ?? "#de5b7b";
        ViewBag.Logo = ParameterManager.GetParam(SkinTheme.LogoTheme.ToString()).Value ?? "Gestor";
        return true;
    }
    if (IsPage)
    {
        Response.Redirect(Url.Action("Login", "Account"));
    }
    return false;
}
```

El método *ValidateSession()* cumple varias funciones en la aplicación y es una de las más importantes en términos de seguridad, pues el objetivo general es validar que el usuario que está intentando ingresar a una vista haya iniciado sesión anteriormente y si tiene los permisos necesarios para permitir ver la vista y devolver los recursos necesarios en ella.

En detalle del método evalúa primero si en la sesión que maneja .NET framework el usuario es diferente de nulo, el cual, indica si un usuario ha iniciado sesión o no, Si el valor es nulo el método devolverá un false o redireccionará a la página de inicio de sesión si se envía un true como parámetro en el método.

Si el valor es diferente de nulo obtiene el objeto donde se guardó la información del usuario, la información que puede tener este usuario es el nombre, los perfiles que maneja y si tiene pendiente cambiar la contraseña. Con estos datos, se obtiene la URL absoluta donde está intentando ingresar el usuario y en el método que se llama *Validate()* se le enviará la ruta y los perfiles, el cual me devolverá un objeto de permisos a los cuales el usuario tiene o no autorización y podrán ser usados en la vista para permitir o no funcionalidades de la vista especifica donde desea ingresar.

También este método devolverá a la vista un booleano con valor *CambioClave*, este es el que determinara si el usuario está obligado a cambiar la contraseña para que en la vista no se muestre el menú de navegación hasta que haga lo que se le solicita. Por otro lado, devolverá otros valores como permisos independientes de botones que tendrá la vista compartida de toda la aplicación (explicación del método más adelante), el color del menú de navegación el cual es un valor parametrizado desde base de datos, el logo y el menú que será mostrado a cada usuario dependiendo de los perfiles y permisos que cada uno tenga el cual es obtenido por el método *GetMenu()* explicado más adelante.

D. Obtener permisos de una página o botón que tenga un usuario.

```
47 referencias
public PermitDTO ValidatePaginaDependiente(string url, List<PerfilDTO> perfiles)
{
    var permit = new PermitDTO();
    var page = Connection.ContextC10.tbl_paginas.FirstOrDefault(p => p.ruta == url);
    if (page != null)
    {
        permit = GetPermissions(perfiles).FirstOrDefault(p => p.IdPagina == page.id_pagina);
    }
    return permit;
}
```

El método *ValidatePaginaDependiente()* recibe como parámetros una URL que será la ruta de la página que se quiere obtener los permisos y una lista de perfiles que será los diferentes perfiles que un usuario puede tener al registrarse en la aplicación. Con estos valores se busca la página en base de datos de la que se tenga alguna coincidencia con la URL recibida y se utiliza el método *GetPermissions()* donde me retorna una lista de páginas en las que el usuario tiene acceso con respecto a los perfiles enviados, el cual, para este método solo importara los permisos de la página que se está solicitando y este será el que se devuelva como valor.

E. Obtener el menú por usuario mostrado en la vista

```
4 referencias
public List<PaginaDTO> GetMenu(List<PerfilDTO> perfiles, List<PermitDTO> perms = null)
{
    if (perms == null)
    {
        perms = GetPermissions(perfiles).Where(p => p.IsGrant).ToList();
    }
    var pages = ListPage().Where(p => perms.Any(pe => pe.IdPagina == p.Id)).ToList();
    return pages;
}
```

El método *GetMenu()* es el encargado en toda la aplicación de devolver las páginas que un usuario podrá navegar dentro de la aplicación de credi10, este recibe como parámetro la lista de los perfiles que tiene el usuario y un parámetro opcional de

enviarle puntualmente los permisos que se le quiera dar a un usuario para devolver las páginas correspondientes. Si este parámetro opcional no se envía se utilizará el método *GetPermissions()* para obtener la lista permisos por página que tenga el valor de *IsGrant* es true, este valor es el que me indica que una página tiene algún tipo de permiso y podrá ser visualizada por el usuario. Con respecto a esta lista se obtendrá ya el objeto con la lista de páginas que serán mostradas en el menú.

F. Obtener lista de páginas con sus permisos respectivos

```
4 referencias
public List<PermitDTO> GetPermissions(List<PerfilDTO> perfiles)
{
    var permissions = new List<PermitDTO>();
    var paginas = Connection.ContextC10.tbl_paginas.ToList();


    foreach (var pag in paginas)
    {
        var permission = new PermitDTO()
        {
            IdPagina = pag.id_pagina,
            Icono = pag.icono,
            Pagina = pag.descripcion
        };

        foreach (var perfil in perfiles)
        {
            if (perfil.Estado)
            {
                var profile = Connection.ContextC10.tbl_perfiles.FirstOrDefault(p => p.id_perfil == perfil.Id);
                var perm = profile.tbl_permisos.FirstOrDefault(p => p.tbl_paginas.id_pagina == pag.id_pagina);
                if (perm != null)
                {
                    permission.Leer = permission.Leer || perm.lectura;
                    permission.Crear = permission.Crear || perm.crear;
                    permission.Actualizar = permission.Actualizar || perm.actualizar;
                    permission.Borrar = permission.Borrar || perm.borrar;
                    permission.Anular = permission.Anular || perm.anular;
                    permission.AnularXDia = permission.AnularXDia || perm.anular_dia;
                }
            }
        }
        permissions.Add(permission);
    }
    return permissions;
}
```

El método *GetPermissions()* recibirá como parámetro una lista de perfiles y obtendrá el listado de todas las páginas y/o botones que están registrados en la base de datos, el método entrara a un ciclo que buscara página por página y a cada página le asignara un objeto que contendrá los permisos de dicha página, los permisos se obtienen recorriendo otro ciclo correspondiente a la lista de perfiles que tenga el usuario y asignando permiso por permiso combinando así los perfiles asignados en uno solo. Al final de los ciclos se obtendrá un objeto de lista de permisos que tendrá asociado a la página y el acceso que tendría por usuario.

G. Seleccionar la oficina mediante la IP de conexión.



```
public OperationResponse GetOficinaIP(string ip)
{
    var result = new OperationResponse();

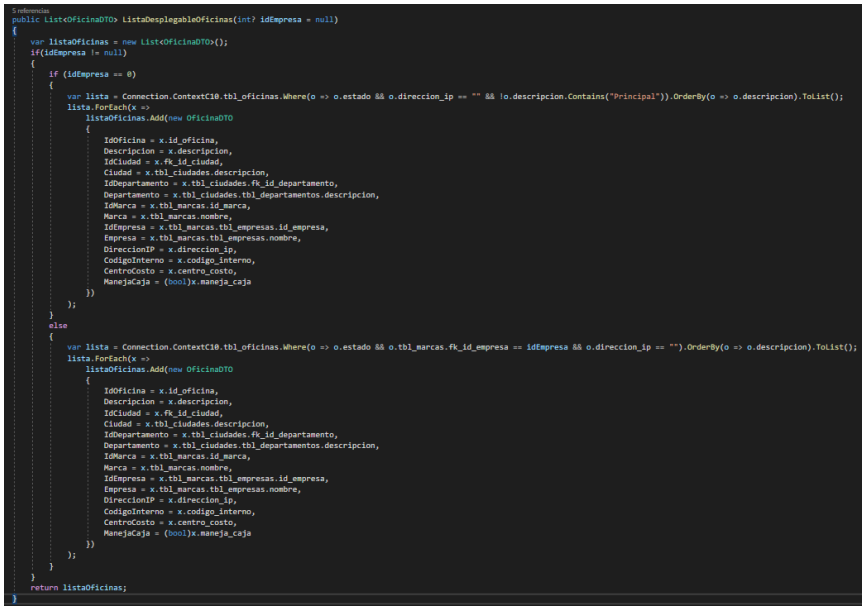
    try
    {
        IPAddress ipAddress = IPAddress.Parse(ip);
        byte[] ipBytes = ipAddress.GetAddressBytes();
        int MascaraOficina = 0;
        int octeto4Ip = 0;
        var octetosIP = ipBytes[0].ToString() + "." + ipBytes[1].ToString() + "." + ipBytes[2].ToString() + ".";

        var oficina = Connection.ContextC10.tbl_oficinas.FirstOrDefault(o => o.estado && o.direccion_ip.Contains(octetosIP));

        if (oficina != null && Int32.TryParse(ipBytes[3].ToString(), out octeto4Ip) && Int32.TryParse(oficina.direccion_ip.Split('/')[1], out MascaraOficina) && octeto4Ip <= MascaraOficina)
        {
            result.Data = oficina.codigo_interno + " - " + oficina.descripcion;
            result.Completed = true;
        }
        else
        {
            result.Completed = false;
        }
        result.Status = true;
    }
    catch (Exception ex)
    {
        result.Message = GetErrorMessage(ex);
        result.IsError = true;
        result.Completed = false;
    }
    return result;
}
```

El método llamado *GetOficinaIP()* es el encargado de validar la IP de conexión del dispositivo y buscar alguna coincidencia en las direcciones IP que tienen asignadas los diferentes puntos de venta. Esta coincidencia se basa separando la IP de conexión con los 3 primeros octetos y buscando oficina por oficina si esta cumple con el valor y está dentro del rango de la máscara que el punto de venta tenga asignado. Si existe alguna coincidencia el método devolver un s*tatus* true con la información del punto de venta encontrado para que en la vista se pueda visualizar.

H. Listar oficinas que no tienen dirección IP fija.



```
public List<OficinaDTO> ListaDesplegableOficinas(int? idEmpresa = null)
{
    var listaOficinas = new List<OficinaDTO>();
    if(idEmpresa != null)
    {
        if (idEmpresa == 0)
        {
            var lista = Connection.ContextC10.tbl_oficinas.Where(o => o.estado && o.direccion_ip == "" && !o.descripcion.Contains("Principal")).OrderBy(o => o.descripcion).ToList();
            lista.ForEach(x =>
                listaOficinas.Add(new OficinaDTO
                {
                    IdOficina = x.id_oficina,
                    Descripcion = x.descripcion,
                    IdCiudad = x.fk_id_ciudad,
                    Ciudad = x.tbl_ciudades.descripcion,
                    IdDepartamento = x.tbl_ciudades.fk_id_departamento,
                    Departamento = x.tbl_ciudades.tbl_departamentos.descripcion,
                    IdMarca = x.tbl_marcas.id_marca,
                    Marca = x.tbl_marcas.nombre,
                    IdEmpresa = x.tbl_marcas.tbl_empresas.id_empresa,
                    Empresa = x.tbl_marcas.tbl_empresas.nombre,
                    DireccionIP = x.direccion_ip,
                    CodigoInterno = x.codigo_interno,
                    CentroCosto = x.centro_costo,
                    ManejaCaja = (bool)x.maneja_caja
                })
            );
        }
        else
        {
            var lista = Connection.ContextC10.tbl_oficinas.Where(o => o.estado && o.tbl_marcas.fk_id_empresa == idEmpresa && o.direccion_ip == "").OrderBy(o => o.descripcion).ToList();
            lista.ForEach(x =>
                listaOficinas.Add(new OficinaDTO
                {
                    IdOficina = x.id_oficina,
                    Descripcion = x.descripcion,
                    IdCiudad = x.fk_id_ciudad,
                    Ciudad = x.tbl_ciudades.descripcion,
                    IdDepartamento = x.tbl_ciudades.fk_id_departamento,
                    Departamento = x.tbl_ciudades.tbl_departamentos.descripcion,
                    IdMarca = x.tbl_marcas.id_marca,
                    Marca = x.tbl_marcas.nombre,
                    IdEmpresa = x.tbl_marcas.tbl_empresas.id_empresa,
                    Empresa = x.tbl_marcas.tbl_empresas.nombre,
                    DireccionIP = x.direccion_ip,
                    CodigoInterno = x.codigo_interno,
                    CentroCosto = x.centro_costo,
                    ManejaCaja = (bool)x.maneja_caja
                })
            );
        }
    }
    return listaOficinas;
}
```

El método llamado *ListDesplegableOficinas()* será el encargado de devolver una lista de oficinas a las vistas, las oficinas que se listan serán solo las que no tienen alguna IP fija asignada en la base de datos y también el listado que se devuelva dependerá del idEmpresa que reciba como parámetro. De esta manera se controla el acceso a los usuarios a diferentes puntos de venta y crear RC.

I. Consultar Pagos pendientes de un cliente

```
1 referencia
public PagosClienteDTO ConsultaPagosPendientes(string numero = "")
{

    var datos = Connection.ContextC10.sp_consulta_pagos_cliente(numero).First();


    var cliente = new PagosClienteDTO();

    if (datos.NombreCompleto == null)
    {
        cliente.StatusCliente = false;
        cliente.Mensaje = Constant.Mensajes.NoExisteCliente;
    }
    else
    {
        if (datos.CreditosVigentes == 0)
        {
            cliente.StatusCliente = false;
            cliente.Mensaje = Constant.Mensajes.SinPagosPendientes;
        }
        else
        {
            cliente.NombreCompleto = CultureInfo.CurrentCulture.TextInfo.ToTitleCase(datos.NombreCompleto.ToLower());
            cliente.CreditosVigentes = datos.CreditosVigentes;
            cliente.StatusCliente = true;
        }
    }

    return cliente;
}
```

El método *ConsultarPagosPendientes()* recibe como parámetro el número de cedula del cliente que quiere consultar la información del pago, este llama un procedimiento almacenado que valida si el cliente ingresado tiene créditos vigentes, si los tiene este devuelve la cantidad de créditos vigentes que tiene, el nombre completo para poder visualizarlo en la vista y un *status* en true para poder continuar con el proceso. Si el cliente no posee créditos vigentes este devolver un *status* en false y un mensaje parametrizado que indica que no tiene ningún pago por realizar.

J. Consulta limite recaudo x empresa

```
1 referencia
public bool ConsultaLimiteRecaudo(int idOficina)
{
    var result = Connection.ContextC10.sp_limite_recaudo_x_empresa(idOficina).First();
    return (bool)result.permisoRecaudo;
}
```

El método *ConsultaLimiteRecaudo()* recibe el id de la oficina que está intentando ingresar a la pantalla para generar un RC, este método consulta un procedimiento almacenado que evalúa si dicha oficina tiene algún limite o no, y si lo tiene evalúa

si este ya se cumplió o puede seguir generando recibos de caja, devolviendo un valor booleano que permite o no esta generación.

K. Validación del recaudo ingresado vs límite de recaudo de la tienda



```
public (bool permiso, string mensaje) ConsultaDetalladaLimiteRecaudo(int idOficina, decimal ValorPago)
{
    var result = Connection.ContextC10.sp_limite_recaudo_x_empresa(idOficina).First();

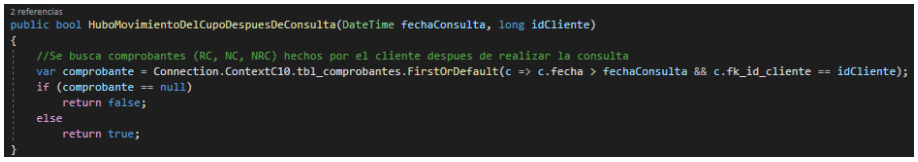
    var mensaje = "";
    var recaudoPermitido = result.limiteRecaudo - result.RC;

    if ((bool)result.permisoRecaudo && ValorPago > recaudoPermitido && recaudoPermitido > 0 )
    {
        result.permisoRecaudo = false;
        mensaje = String.Format(ParameterManager.GetParamsms(Constant.MensajesKey.PagoSuperaLimiteRecaudo).Value, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, recaudoPermitido));
    }
    else
    {
        mensaje = ParameterManager.GetParamsms(Constant.MensajesKey.LimiteRecaudo).Value;
    }

    return ((bool)result.permisoRecaudo, mensaje);
}
```

El método *ConsultaDetalladaLimiteRecaudo()* es una validación que se hará en el último momento cuando ya el usuario dio clic en aplicar pago, este recibe como parámetros el id de la oficina y el valor del pago que ingreso en pantalla y evalúa si este valor excede o no el límite de recaudo que pueda recibir el punto de venta. Si este es mayor cancelara el pago y avisara al usuario sobre el límite que alcanzo. Si el valor es menor al límite este devolverá un valor verdadero en el permiso para que continue el procesamiento del pago.

L. Validación de movimientos del crédito entre consulta y aplicación del pago



```
2 referencias
public bool HuboMovimientoDelCupoDespuesDeConsulta(DateTime fechaConsulta, long idCliente)
{
    //Se busca comprobantes (RC, NC, NRC) hechos por el cliente despues de realizar la consulta
    var comprobante = Connection.ContextC10.tbl_comprobantes.FirstOrDefault(c => c.fecha > fechaConsulta && c.fk_id_cliente == idCliente);
    if (comprobante == null)
        return false;
    else
        return true;
}
```

El método *HuboMovimientoDelCupoDespuesDeConsulta()* es llamado en el último momento antes de aplicar el pago, el método recibe como parámetro la fecha de consulta y el id del cliente con el que están haciendo el proceso. Después de consultar y recibir información puntual al cliente de lo que debía, este valor no hubiera cambiado por un movimiento realizado durante el tiempo que ha transcurrido hasta ese momento. Un movimiento puede ser una NC, una RC o una NFR y estos muy probablemente cambien el valor que el cliente debe pagar. Esto es una validación que se agregó para evitar errores en la aplicación de los pagos y NFR y que las transacciones a realizar no se vean alteradas por datos que pueden variar en segundos.

M. Obtener los datos básicos del cliente

```
3 referencias
public ClienteDTO GetDataClientes(string numeroIdentificacion)
{
    var infoCliente = Connection.ContextC10.sp_get_datos_clientes(numeroIdentificacion).First();
    var clientes = CustomParser.Convert(infoCliente);
    return clientes;
}
```

El método *GetDataClientes()* recibe como parámetro el número de identificación y este hará uso de un procedimiento almacenado que traerá los datos básicos del cliente como lo son el nombre, numero de celular, correo electrónico y la foto en caso de que la tenga. Estos datos serán usados en la vista o en el proceso siguiente de aplicar el pago y poder notificarlo.

N. Crear Ticket informativo del pago.

```
3 referencias
public static OperationResponse CrearTicketPago(Ticket infoTicket, ControllerContext context = null)
{
    var result = new OperationResponse();
    try
    {

        string customSwitches = string.Format(
                            "--header-html  \"{0}\" " +
                            "--header-spacing \"5\" " +
                            "--footer-spacing \"10\" " +
                            "--footer-font-size \"10\" " +
                            "--header-font-size \"10\" " +
                            "--footer-line --footer-font-size \"9\" " +
                            "--footer-right \" [page]/[toPage]\" " +
                            "--footer-font-name \"Times New Roman\" " +
                            "--header-font-name \"Times New Roman\" "  ,
                            "");

        var a = new ViewAsPdf();
        a.ViewName = "ticketPago";
        a.CustomSwitches = customSwitches;
        a.Model = infoTicket;

        if(infoTicket.detallePago.Count < 5)
            a.PageHeight = 150.0;
        else if(infoTicket.detallePago.Count >= 5 && infoTicket.detallePago.Count < 10)
            a.PageHeight = 170.0;
        else
            a.PageHeight = 220.0;

        a.PageWidth = 80.0;
        a.PageMargins = new Rotativa.Options.Margins(5, 0, 0, 5);

        // Create an instance of your controller class (in example, I guess you are using mentioned quotesController)

        // Generate controller context, to use in the next step
        var pdfBytes = a.BuildFile(context);
        //Creacion de pdf en memoria
        var file = new MemoryStream();
        file.Write(pdfBytes, 0, pdfBytes.Length);
        file.Close();

        byte[] fileB64 = file.ToArray();

        //Por si se quiere crear fisicamente el archivo pdf
        ///var fileStream = new FileStream(path, FileMode.Create);
        ///fileStream.Write(pdfBytes, 0, pdfBytes.Length);
        ///fileStream.Close();

        result.Data = Convert.ToBase64String(fileB64);

        result.Status = true;
        result.Completed = true;
        result.Message = "Se creo el documento exitosamente";
    }
    catch (Exception ex)
    {
        result.Status = false;
        result.Completed = false;
        result.Message = ex.Message;
    }
    return result;
}
```

Método que crea el ticket con la información del pago realizado en pdf, para crear el documento se utiliza la librería de rotativa wkhtmltopdf, el cual se necesita de una vista como plantilla para crear el pdf y un contexto que tendrá la información necesaria de donde está ubicada la vista para su procesamiento. El método además recibe el objeto infoTicket que tendrá toda la información del cliente y el detalle de cada pago para su correcta creación. El pdf se retorna como un string del documento convertido a Base64.

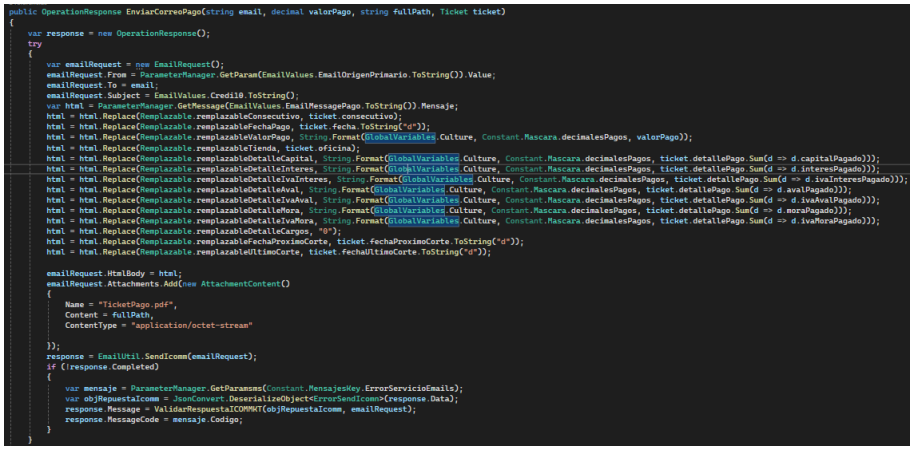
O. Notificar por SMS el pago

```
2 referencias
public OperationResponse EnviarSMSPago(decimal valorPago, string celular)
{
    var response = new OperationResponse();
    try
    {
        var smsRequest = new SmsRequest();
        smsRequest.telephone = "9" + celular;
        var msg = ParameterManager.GetMessage(SmsValues.SmsPago.ToString()).Mensaje;
        smsRequest.message = string.Format(msg, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, valorPago), DateTime.Now.ToString("d"));

        response = SmsUtil.Send(smsRequest);
    }
    catch (DbEntityValidationException ex)
    {
        response.Message = GetErrorMessage(ex);
        response.IsError = true;
        response.Status = false;
        response.Completed = false;
    }
    catch (Exception ex)
    {
        response.Message = GetErrorMessage(ex);
        response.IsError = true;
        response.Status = false;
        response.Completed = false;
    }
    return response;
}
```

Método encargado de construir el request y posterior envió del SMS informativo del pago realizado en la tienda o en el link de recaudo, para él envió se consume el API proporcionado por Wolkvox.

P. Notificar por Correo el pago



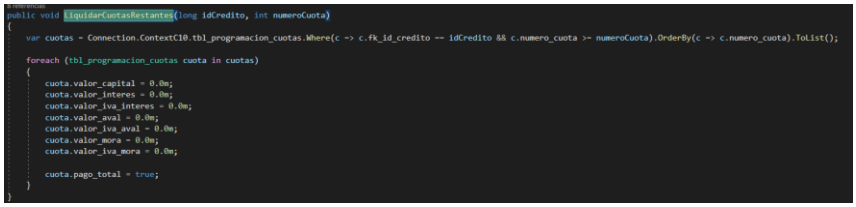
```
public OperationResponse EnviarCorreoPago(string email, decimal valorPago, string fullPath, Ticket ticket)
{
    var response = new OperationResponse();
    try
    {
        var emailRequest = new EmailRequest();
        emailRequest.From = ParameterManager.GetParam(EmailValues.EmailOrigenPrimario.ToString()).Value;
        emailRequest.To = email;
        emailRequest.Subject = EmailValues.Credi10.ToString();
        var html = ParameterManager.GetMessage(EmailValues.EmailMessagePago.ToString()).Mensaje;
        html = html.Replace(Remplazable.remplazableConsecutivo, ticket.consecutivo);
        html = html.Replace(Remplazable.remplazableFechaPago, ticket.fecha.ToString("d"));
        html = html.Replace(Remplazable.remplazableValorPago, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, valorPago));
        html = html.Replace(Remplazable.remplazableTienda, ticket.oficina);
        html = html.Replace(Remplazable.remplazableDetalleCapital, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, ticket.detallePago.Sum(d => d.capitalPagado)));
        html = html.Replace(Remplazable.remplazableDetalleInteres, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, ticket.detallePago.Sum(d => d.interesPagado)));
        html = html.Replace(Remplazable.remplazableDetalleIvaInteres, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, ticket.detallePago.Sum(d => d.ivaInteresPagado)));
        html = html.Replace(Remplazable.remplazableDetalleAval, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, ticket.detallePago.Sum(d => d.avalPagado)));
        html = html.Replace(Remplazable.remplazableDetalleIvaAval, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, ticket.detallePago.Sum(d => d.ivaAvalPagado)));
        html = html.Replace(Remplazable.remplazableDetalleMora, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, ticket.detallePago.Sum(d => d.moraPagado)));
        html = html.Replace(Remplazable.remplazableDetalleIvaMora, String.Format(GlobalVariables.Culture, Constant.Mascara.decimalesPagos, ticket.detallePago.Sum(d => d.ivaMoraPagado)));
        html = html.Replace(Remplazable.remplazableDetalleCargos, "0");
        html = html.Replace(Remplazable.remplazableFechaProximoCorte, ticket.fechaProximoCorte.ToString("d"));
        html = html.Replace(Remplazable.remplazableUltimoCorte, ticket.fechaUltimoCorte.ToString("d"));

        emailRequest.HtmlBody = html;
        emailRequest.Attachments.Add(new AttachmentContent()
        {
            Name = "TicketPago.pdf",
            Content = fullPath,
            ContentType = "application/octet-stream"

        });
        response = EmailUtil.SendIcomm(emailRequest);
        if (!response.Completed)
        {
            var mensaje = ParameterManager.GetParamsms(Constant.MensajesKey.ErrorServicioEmails);
            var objRepuestaIcomm = JsonConvert.DeserializeObject<ErrorSendIcomm>(response.Data);
            response.Message = ValidarRespuestaICOMMKT(objRepuestaIcomm, emailRequest);
            response.MessageCode = mensaje.Codigo;
        }
    }
```

Método encargado de construir el request y posterior envió del correo informativo del pago realizado en la tienda o en el link de recaudo, junto a su tirilla de pago enviado como un documento adjunto, para él envió se consume el API proporcionado por Icommarkeitng.

Q. Liquidar el crédito



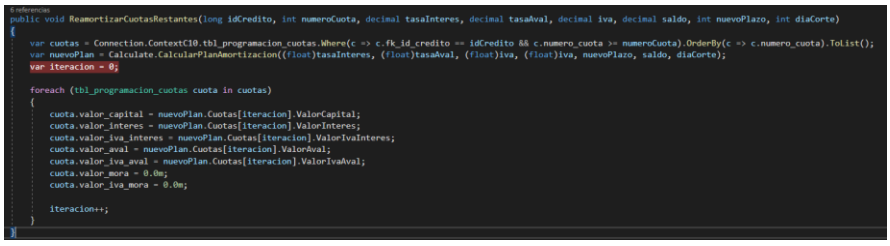
```
public void LiquidarCuotasRestantes(long idCredito, int numeroCuota)
{
    var cuotas = Connection.ContextC10.tbl_programacion_cuotas.Where(c => c.fk_id_credito == idCredito && c.numero_cuota >= numeroCuota).OrderBy(c => c.numero_cuota).ToList();

    foreach (tbl_programacion_cuotas cuota in cuotas)
    {
        cuota.valor_capital = 0.0m;
        cuota.valor_interes = 0.0m;
        cuota.valor_iva_interes = 0.0m;
        cuota.valor_aval = 0.0m;
        cuota.valor_iva_aval = 0.0m;
        cuota.valor_mora = 0.0m;
        cuota.valor_iva_mora = 0.0m;

        cuota.pago_total = true;
    }
}
```

Al llamarse pondrá todos los valores en 0 de las cuotas no presupuestadas de un crédito, que se supone que se canceló en su totalidad en el proceso del pago.

R. Reamortizar el crédito



```
public void ReamortizarCuotasRestantes(long idCredito, int numeroCuota, decimal tasaInteres, decimal tasaAval, decimal iva, decimal saldo, int nuevoPlazo, int diaCorte)
{
    var cuotas = Connection.ContextC10.tbl_programacion_cuotas.Where(c => c.fk_id_credito == idCredito && c.numero_cuota >= numeroCuota).OrderBy(c => c.numero_cuota).ToList();
    var nuevoPlan = Calculate.CalcularPlanAmortizacion((float)tasaInteres, (float)tasaAval, (float)iva, (float)iva, nuevoPlazo, saldo, diaCorte);
    var iteracion = 0;

    foreach (tbl_programacion_cuotas cuota in cuotas)
    {
        cuota.valor_capital = nuevoPlan.Cuotas[iteracion].ValorCapital;
        cuota.valor_interes = nuevoPlan.Cuotas[iteracion].ValorInteres;
        cuota.valor_iva_interes = nuevoPlan.Cuotas[iteracion].ValorIvaInteres;
        cuota.valor_aval = nuevoPlan.Cuotas[iteracion].ValorAval;
        cuota.valor_iva_aval = nuevoPlan.Cuotas[iteracion].ValorIvaAval;
        cuota.valor_mora = 0.0m;
        cuota.valor_iva_mora = 0.0m;

        iteracion++;
    }
}
```

Método que recibe un saldo, un nuevo plazo, además de los valores de las tasas en que fue desembolsado un crédito para modificar la programación de cuotas en base al plazo original y al nuevo saldo de capital que se encuentra.

S. ValidarInformacionDelArchivoExcel

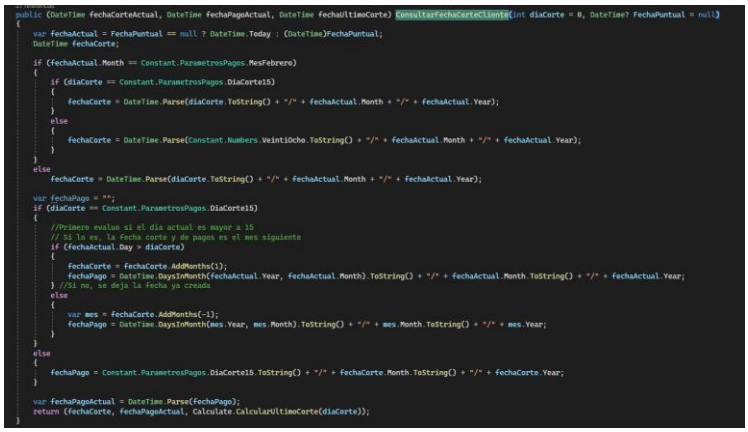


```
1 referencia
public bool ValidarInformacionDelArchivoExcel(DateTime Fecha, string Cedula, string Valor, string Banco, string Medio)
{
    var conversion = 0;
    var Bancos = Connection.ContextC10.tbl_parametros_pagos.Where(p => p.grupo == "Banco").ToList();
    var Medios = Connection.ContextC10.tbl_parametros_pagos.Where(p => p.grupo == "MedioPago").ToList();

    if (!int.TryParse(Cedula, out conversion))
    {
        return false;
    }
    if (!int.TryParse(Valor, out conversion))
    {
        return false;
    }
    if (Bancos.FirstOrDefault(p=>p.valor == Banco.ToUpper()) == null)
    {
        return false;
    }
    if (Medios.FirstOrDefault(p => p.valor == Medio.ToUpper()) == null)
    {
        return false;
    }
    else
        return true;
}
```

Método que valida unas condiciones en los valores que debería contener el archivo de Excel que están subiendo, estas aseguran que la cedula y el pago sean un valor numérico, que la información del banco y el medio estén registrados en el maestro de parámetros pagos y si todas las validaciones cumplen se envía verdadero para guardar el registro o la fila analizada, si una de estas no cumple se retorna un valor falso para ignorar dicho registro.

T. **Consultar fecha corte cliente**



```
public (DateTime fechaCorteActual, DateTime fechaPagoActual, DateTime fechaUltimoCorte) ConsultarFechaCorteCliente(int diaCorte = 0, DateTime? FechaPuntual = null)
{
    var fechaActual = FechaPuntual == null ? DateTime.Today : (DateTime)FechaPuntual;
    DateTime fechaCorte;

    if (fechaActual.Month == Constant.ParametrosPagos.MesFebrero)
    {
        if (diaCorte == Constant.ParametrosPagos.DiaCorte15)
        {
            fechaCorte = DateTime.Parse(diaCorte.ToString() + "/" + fechaActual.Month + "/" + fechaActual.Year);
        }
        else
        {
            fechaCorte = DateTime.Parse(Constant.Numbers.VeintiOcho.ToString() + "/" + fechaActual.Month + "/" + fechaActual.Year);
        }
    }
    else
        fechaCorte = DateTime.Parse(diaCorte.ToString() + "/" + fechaActual.Month + "/" + fechaActual.Year);

    var fechaPago = "";
    if (diaCorte == Constant.ParametrosPagos.DiaCorte15)
    {
        //Primero evalua si el dia actual es mayor a 15
        // Si lo es, la fecha corte y de pagos es el mes siguiente
        if (fechaActual.Day > diaCorte)
        {
            fechaCorte = fechaCorte.AddMonths(1);
            fechaPago = DateTime.DaysInMonth(fechaActual.Year, fechaActual.Month).ToString() + "/" + fechaActual.Month.ToString() + "/" + fechaActual.Year;
        } //Si no, se deja la fecha ya creada
        else
        {
            var mes = fechaCorte.AddMonths(-1);
            fechaPago = DateTime.DaysInMonth(mes.Year, mes.Month).ToString() + "/" + mes.Month.ToString() + "/" + mes.Year;
        }
    }
    else
    {
        fechaPago = Constant.ParametrosPagos.DiaCorte15.ToString() + "/" + fechaCorte.Month.ToString() + "/" + fechaCorte.Year;
    }

    var fechaPagoActual = DateTime.Parse(fechaPago);
    return (fechaCorte, fechaPagoActual, Calculate.CalcularUltimoCorte(diaCorte));
}
```

El método recibe dos parámetros, un día de corte que para este momento el cliente ya debería tener algún valor y una fecha que por defecto es nulo, el método retornara los valores de la fecha de corte, la fecha máxima de pago y la última fecha

de corte que ha tenido el cliente con respecto al día de corte recibido y la fecha actual o la recibida como parámetro.

Documentación de Arquitectura

Log

Comentado [HA5]: Agregar el detalle de lo que se registra en el log

Los logs que se registran en el módulo de pagos para seguir una auditoria son los siguientes:

Acción	Resultado	Detalle
Consulta Pago del cliente	Vista con la información del cliente	Filtro ingresado en la consulta
Aplicar Pago	Información del resultado al intentar aplicar el pago	Se guarda la información del cliente, cuanto era su pago mínimo y total, la tienda donde está realizando el proceso y cuanto fue el valor del pago.
Consulta Pago por Consignación	Vista con la información del cliente	Filtro ingresado en la consulta
Cargue Archivo Pagos	Resultado al intentar guardar los registros del archivo	Sin detalle
Reprocesar Pago	Resultado al intentar aplicar un pago individual de los subidos en el archivo	Id del registro que se intentó reprocesar el pago.
Cargue Archivo Pagos PayU	Resultado al intentar guardar los registros del archivo	Sin detalle
Consulta Estado Cargue Archivo	No aplica	Sin detalle
Consulta Detalle Cargue Archivo	No aplica	Id del cargue subido.

Todos los logs se registran con el nombre de usuario que está realizando la acción, la IP del dispositivo donde se está conectando y la fecha y hora en que ha realizado las consultas.

Lista de Programas, archivos, paquetes

- **sp_get_cargue_pagos:**

Procedimiento almacenado que devuelve la lista de todos los cargues realizados ordenado de manera descendente por fecha.

- **sp_consulta_pagos_cliente:**

Procedimiento almacenado que valida si el cliente tiene créditos pendientes por pagar y si lo tiene retorna el nombre y número de créditos vigentes.

- **sp_limite_recaudo_x_empresa:**

Procedimiento almacenado que al enviarle una tienda devuelve si la empresa a la que pertenece dicha tienda alcanzo o cuanto le falta por alcanzar el límite del recaudo.

- **sp_actualizacion_vigencia_credito:**

Procedimiento almacenado que valida si un crédito ya fue pagado en su totalidad para cambiarle el estado correspondiente.

- **sp_disminuir_y_activar_cupo:**

Procedimiento almacenado que se ejecuta al terminar un pago, si este cancelo por completo su deuda y el cliente tiene el cupo bloqueado por casa de cobranza este se le activara el cupo nuevamente con una disminución parametrizada desde BD.

- **sp_get_datos_clientes:**

Procedimiento almacenado que trae los datos actualizados como foto, celular, correo electrónico y nombre completo del cliente que están consultando para aplicar un pago.

- **sp_validar_vigencia_credito:**

Procedimiento almacenado que valida si un crédito se encuentra bien amortizado o si este ya se encuentra pagado pero su estado sigue vigente, al encontrar estos casos que en su mayoría son datos que vienen migrados se corrigen dentro del procedimiento.

- **sp_pagos_pendientes:**

Procedimiento almacenado que retorna los valores pendientes para pagar de una cuota en específico que tiene un abono en ella.

- **sp_consulta_abono_cuota:**

Procedimiento almacenado que consulta el detalle de cómo se realizó un abono a una cuota en específico.

- **sp_consulta_pagos_anticipados:**

Procedimiento almacenado que valida si un pago anterior le fue cobrado un valor anticipado para tener en cuenta en el pago actual que se está realizando.

- **sp_existe_registro_pago:**

Procedimiento almacenado que valida que un registro que se esté cargando en un archivo no haya sido subido anteriormente, este caso se detecta si todos los valores en una fila son iguales a el nuevo registro que están intentando subir,

- **Lista de Script´s y la descripción de lo que contiene cada uno.**

Script 1. Creación de tablas: Para crear el esquema completo de la base de datos de la aplicación.

Creacion_BD_C10_Credi10Net.sql

1.4 Observaciones y/o Comentarios

Hay casos en que un crédito no se le ve reflejado intereses, avales ni mora en todas sus cuotas o en algunas de ellas, esto es probable ya que el cupo tiene un estado el cual se le congela la facturación por completo y estos conceptos se les dejara de cobrar en el futuro para evitar pérdidas en credi10.

Otro caso que se verá reflejado en el pago es que si un cupo llega a sobrepasar los noventa (90) días de mora se le cesara la facturación, pero no los conceptos a cobrar, se le seguirá presupuestando los conceptos de interés y aval y al momento de que el cliente pague estos conceptos, se hará el proceso de facturación en el corte siguiente al pago ya que efectivamente fueron cobrados.

2. CASOS DE PRUEBAS UNITARIAS

A continuación, se demuestra las pruebas funcionales hechas en el ambiente de pruebas.

- **Pago en tienda**

Se ingresa la identificación del cliente al que se quiere realizar el pago y la tienda donde se quiere aplicar.



En este punto se evidencia los datos del cliente y la tienda en donde están realizando el pago junto al pago mínimo y total que tiene el cliente en la fecha de consulta. Se escoge una opción para el pago y se da clic en el botón de aplicar pago.



Después de dar clic se mostrará un mensaje de confirmación del pago realizado y se abrirá una nueva ventana con el ticket informativo de cómo se aplicó el pago en los diferentes items que se le fueran a cobrar.



Dicho ticket podrá ser impreso y será el que se le enviará por correo electrónico al cliente para confirmar el pago realizado.

Santiago de Cali Fecha: 26/01/2024 3:02 p. m.
STF GROUP S.A.
805003626

STUDIO F Studio F Multicentro Ibague
CC Multicentro Ibague Cra 5 No 60123
4851515

Datos del Recibo
Nro Id:
1113652660
Nombre Cliente:
Claudia Alvarez Gomez
Recibo de Caja
RC10065

Detalle del Pago

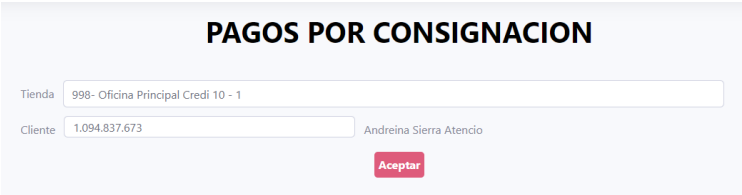
Valor Total
249.934

Cap.	Int.	Mora	Aval	Iva
248.255	0	1.411	0	268

Fecha Impresión: 26/01/2024 3:02:45 p. m.

- Pagos por consignación

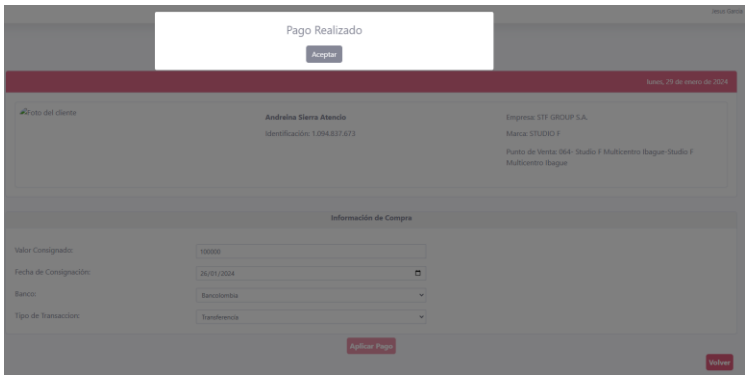
Se ingresa la identificación del cliente al que se quiere realizar el pago y la tienda creada para recibir estos pagos.



Si el cliente tiene saldo pendiente le habilita el botón de aceptar para seguir a la otra pantalla con la información del cliente y un formulario para ingresar los datos de la consignación reportada. Al ingresar los datos se le da en el botón de Aplicar Pago.



Después de aplicar el pago se le muestra al usuario el mensaje de que el pago fue aplicado y al darle aceptar le recarga la página. El pago por este medio no genera ticket ni informa al cliente del pago aplicado.



- Cargue de consignaciones

Estos serán los registros del Excel para subir, correspondiente a un pago que no se haya aplicado en el sistema.

A	B	C	D	E
FECHA	CEDULA	VALOR	BANCO	MEDIO
6/01/2023	1107089192	10000	BANCOLOMBIA	CG

Se carga el archivo y se digita el punto de venta en donde se quiere asociar el pago.



Se muestra el mensaje de que el archivo fue subido exitosamente y nos muestra el id del proceso con el que fue cargado para su posterior seguimiento.



- Cargue de Pagos PayU

Estos serán los registros del Excel para subir, correspondiente a un pago que no se haya aplicado en el sistema.

A	B	C	D
Cedula	**ValorPago**	**Fecha Pago M/DD/AAAA**	**Medio de Pago**
66782382	120000	09/27/2023	PSE



En esta función solo se debe cargar el archivo y darle al botón guardar, si el registro fue subido exitosamente se mostrará el mensaje y el id del proceso con el que quedo guardado para su posterior seguimiento.



En la vista de Seguimiento se puede evidenciar la información del cargue, la cantidad de registros subidos, la cantidad de pagos que fueron aplicados correctamente y la cantidad de registros donde hubo errores en el proceso.



1	Aplicar pago en tienda	Aplico el pago y descontó el saldo de la deuda	SI
2	Aplicar pago por consignación	Aplico el pago y descontó el saldo de la deuda	SI
3	Subir Cargue de consignacion	Que retorne el resultado de la subida y el id del proceso para el seguimiento	SI
4	Subir cargue de PayU	Que retorne el resultado de la subida y el id del proceso para el seguimiento	SI